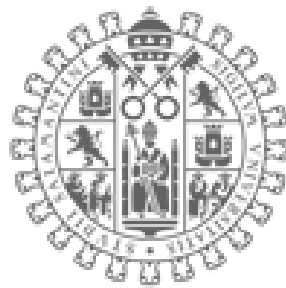


Calidad y Auditoría

Herramientas de Auditoría Software



**VNiVERSiDAD
D SALAMANCA**

CAMPUS OF INTERNATIONAL EXCELLENCE

Miguel Barranquero Diez

Índice

1. Introducción	4
2. OpenVAS	5
2.1. Características Principales	5
2.2. Ventajas y Desventajas	6
2.3. OpenVAS - Caso Práctico	7
2.3.1. Descripción del Caso	7
2.3.2. Entorno de Pruebas	7
2.3.3. Acceso a la Plataforma	7
2.3.4. Panel de Control Principal	8
2.3.5. Creación de Nueva Tarea de Escaneo	9
2.3.6. Vista de Tareas Configuradas	9
2.3.7. Resultados del Escaneo	10
2.3.8. Detalle de Vulnerabilidad Específica	11
2.3.9. Generación de Informes	12
2.3.10. Conclusiones del Caso Práctico	13
3. WireShark	14
3.1. Características Principales	14
3.2. Ventajas y Desventajas	14
3.3. Wireshark - Caso Práctico	14
3.3.1. Descripción del Caso	15
3.3.2. Entorno de Pruebas	15
3.3.3. Captura y Visualización del Tráfico	15
3.3.4. Estructura de un Paquete de Red	15
3.3.5. Análisis de Protocolo DNS	16
3.3.6. Detalle de Resolución DNS	17
3.3.7. Análisis de Tráfico HTTP	17
3.3.8. Estructura de una Petición HTTP	18
3.3.9. Reconstrucción de Conversaciones TCP	19
3.3.10. Estadísticas de la Captura	19
3.3.11. Conclusiones del Caso Práctico	20
4. Lynis	21
4.1. Características Principales	21
4.2. Ventajas y Desventajas	21
4.3. Lynis - Caso Práctico	22
4.3.1. Descripción del Caso	22
4.3.2. Entorno de Pruebas	22
4.3.3. Instalación y Verificación	22
4.3.4. Ejecución de la Auditoría	22
4.3.4.1. Inicialización del Escaneo	22
4.3.4.2. Análisis del Kernel y Hardening	23
4.3.4.3. Auditoría de Usuarios y Autenticación	24
4.3.4.4. Análisis de Sistemas de Ficheros	25
4.3.4.5. Warnings y Sugerencias Detectadas	25
4.3.5. Índice de Endurecimiento (Hardening Index)	26

4.3.6. Conclusiones del Caso Práctico	27
5. OWASP ZAP (Zed Attack Proxy)	29
5.1. Características Principales	29
5.2. Ventajas y Desventajas	30
5.3. OWASP ZAP - Caso Práctico	31
5.3.1. Descripción del Caso	31
5.3.2. Entorno de Pruebas	31
5.3.3. Inicialización de OWASP ZAP	31
5.3.4. Configuración del Escaneo	31
5.3.5. Fase de Spider (Rastreo Web)	32
5.3.6. Estructura del Sitio Web	33
5.3.7. Escaneo Activo de Vulnerabilidades	33
5.3.8. Análisis de Alertas Detectadas	34
5.3.9. Generación del Informe de Auditoría	35
5.3.10. Informe HTML Generado	36
Bibliografía	37

1. Introducción

En el contexto actual de transformación digital, donde las organizaciones dependen cada vez más de sus sistemas de información para operar, tomar decisiones y generar valor, la auditoría informática y el control interno se han convertido en pilares fundamentales para garantizar la seguridad, integridad y confiabilidad de la infraestructura tecnológica. La creciente complejidad de los entornos TI, sumada al incremento de amenazas cibernéticas y a la exigencia de cumplimiento normativo, demanda herramientas especializadas que permitan evaluar, monitorizar y asegurar el correcto funcionamiento de los sistemas.

Las técnicas de auditoría asistida por computador (CAAT, por sus siglas en inglés) y el software de auditoría generalizado representan recursos indispensables para los profesionales de la auditoría y la seguridad informática. Estas herramientas facilitan la identificación de vulnerabilidades, el análisis de tráfico de red, la evaluación de configuraciones de seguridad y la detección de anomalías, proporcionando evidencia objetiva y cuantificable que sustenta la toma de decisiones estratégicas en materia de seguridad y control interno.

El presente trabajo tiene como objetivo analizar y presentar cuatro herramientas representativas del ecosistema actual de auditoría informática, abarcando diferentes ámbitos de aplicación y aproximaciones técnicas. Se ha seleccionado un conjunto diverso que incluye soluciones de código abierto bajo licencia GNU/GPL, garantizando así su accesibilidad y transparencia. Las herramientas elegidas son:

- **OpenVAS** (Open Vulnerability Assessment System): plataforma integral de escaneo y gestión de vulnerabilidades que permite identificar debilidades de seguridad en sistemas y redes mediante una extensa base de datos actualizada diariamente.
- **Wireshark**: analizador de protocolos de red líder en la industria, utilizado para la captura y examen detallado del tráfico de datos, facilitando el diagnóstico de problemas de red y la detección de actividades sospechosas.
- **Lynis**: herramienta de auditoría de seguridad y hardening para sistemas Unix/Linux, que evalúa la configuración del sistema operativo y proporciona recomendaciones concretas para mejorar la postura de seguridad.
- **OWASP ZAP** (Zed Attack Proxy): proxy de interceptación especializado en seguridad de aplicaciones web, diseñado para identificar vulnerabilidades en aplicaciones web mediante técnicas de escaneo automatizado y manual.

Para cada herramienta se presenta una descripción técnica detallada que incluye sus características principales, ventajas y limitaciones, información sobre su licenciamiento y disponibilidad, así como un caso práctico documentado con capturas de pantalla que ilustra su aplicación en escenarios reales de auditoría. Esta aproximación práctica permite no solo comprender las capacidades teóricas de cada solución, sino también visualizar su implementación concreta y los resultados que pueden obtenerse.

El análisis comparativo de estas cuatro herramientas proporciona una visión integral de las posibilidades actuales en el campo de la auditoría informática asistida por tecnología, demostrando cómo diferentes enfoques y especializaciones se complementan para construir una estrategia de seguridad robusta y exhaustiva. A través de este trabajo, se evidencia que la combinación adecuada de herramientas CAAT permite abordar de manera efectiva los múltiples desafíos que enfrentan las organizaciones en la protección de sus activos digitales y el cumplimiento de sus objetivos de control interno.

2. OpenVAS

En el ámbito de la seguridad informática y la gestión de vulnerabilidades, la evaluación proactiva de los sistemas es una actividad fundamental para garantizar la protección de la infraestructura tecnológica ante posibles amenazas. En este contexto, OpenVAS (Open Vulnerability Assessment System) se consolida como una de las herramientas más reconocidas y potentes para el escaneo y análisis de vulnerabilidades, permitiendo a administradores, analistas de seguridad y auditores identificar debilidades antes de que puedan ser explotadas por actores maliciosos.

OpenVAS es una herramienta de código abierto diseñada para detectar y evaluar vulnerabilidades de seguridad en sistemas informáticos y redes, ofreciendo una visión profunda de las debilidades presentes en la infraestructura. Permite inspeccionar configuraciones, identificar software desactualizado, detectar contraseñas débiles y descubrir posibles puntos de entrada para ataques.

2.1. Características Principales

La herramienta OpenVAS destaca por las siguientes funcionalidades clave:

1. **Código abierto y gratuito:** Disponible bajo licencia GNU GPL, mantenido por Greenbone Networks desde 2006.
2. **Base de datos exhaustiva:** Dispone de más de 50.000 pruebas de vulnerabilidades (NVT) en la versión comunitaria gratuita y hasta 100.000 en la versión comercial, cubriendo más de 26.000 CVEs. La base de datos se actualiza diariamente.
3. **Escaneo autenticado y no autenticado:** Permite análisis desde dos perspectivas: no autenticado (visión externa del atacante) y autenticado (análisis profundo con credenciales SSH, RDP, etc.).
4. **Compatibilidad multiplataforma:** Soporta protocolos de alto y bajo nivel, tanto de Internet como industriales.
5. **Interfaz gráfica web (GSA):** Greenbone Security Assistant facilita la configuración, programación y generación de informes.
6. **Interfaz de línea de comandos (GVM-CLI):** Herramienta CLI para automatización y operaciones avanzadas.
7. **Gestión centralizada:** OpenVAS Manager actúa como intermediario entre el escáner y las interfaces, gestionando tareas y resultados.
8. **Optimización para gran escala:** Capacidades de ajuste para infraestructuras extensas sin comprometer efectividad.
9. **Lenguaje interno programable:** Permite implementar pruebas personalizadas mediante un potente lenguaje interno.
10. **Informes detallados:** Genera reportes completos con criticidad, descripción técnica, impacto y recomendaciones de remediación.
11. **Priorización de riesgos:** Clasifica vulnerabilidades según severidad para optimizar recursos.
12. **Cumplimiento normativo:** Facilita el cumplimiento de estándares como PCI DSS con documentación para auditorías.

2.2. Ventajas y Desventajas

Entre sus ventajas, OpenVAS sobresale por ser una solución completa, gratuita y de alto nivel profesional para la gestión de vulnerabilidades. Su extensa base de datos, actualizada diariamente, garantiza la detección de amenazas recientes. La capacidad de realizar escaneos externos e internos autenticados proporciona una evaluación integral. Su naturaleza de código abierto permite revisar plugins para identificar falsos positivos y contribuir a la mejora continua. La comunidad activa ofrece soporte rápido y conocimiento compartido. La arquitectura modular (escáner, gestor, interfaces web y CLI) ofrece flexibilidad para adaptarse desde pequeñas empresas hasta grandes corporaciones. La programación automática de escaneos facilita la integración en procesos de seguridad continua y DevSecOps. Como complemento a medidas preventivas tradicionales, OpenVAS proporciona una capa adicional de defensa mediante identificación proactiva.

No obstante, presenta ciertos inconvenientes: requiere conocimientos técnicos avanzados en seguridad informática, des y sistemas para interpretar correctamente los resultados. La configuración inicial puede resultar compleja, especialmente al instalar desde código fuente. En entornos grandes con alta diversidad de activos, la gestión puede demandar recursos significativos de sistema y personal especializado. Como cualquier escáner, puede generar falsos positivos que requieren validación manual. Aun así, OpenVAS sigue siendo una herramienta imprescindible para la auditoría, evaluación de vulnerabilidades y mejora continua de la seguridad, combinando potencia, precisión, flexibilidad y un modelo completamente abierto que fomenta la transparencia y colaboración en la comunidad de seguridad informática.

2.3. OpenVAS - Caso Práctico

2.3.1. Descripción del Caso

El objetivo de este caso práctico es realizar un escaneo de vulnerabilidades sobre un sistema local utilizando OpenVAS (actualmente conocido como Greenbone Vulnerability Management). La herramienta analizará el sistema en busca de fallos de seguridad conocidos, configuraciones inseguras y software desactualizado que podría ser explotado por atacantes.

2.3.2. Entorno de Pruebas

- **Herramienta:** OpenVAS / Greenbone Vulnerability Management 26.1.0
- **Plataforma:** Contenedor Docker (immauss cybersecurity)
- **Objetivo del escaneo:** 127.0.0.1 (localhost)
- **Fecha de análisis:** 6 de noviembre de 2025
- **Total de vulnerabilidades detectadas:** 56 hallazgos

2.3.3. Acceso a la Plataforma

OpenVAS proporciona una interfaz web completa para gestionar escaneos de seguridad. La Figure 1 muestra la pantalla de acceso al sistema.

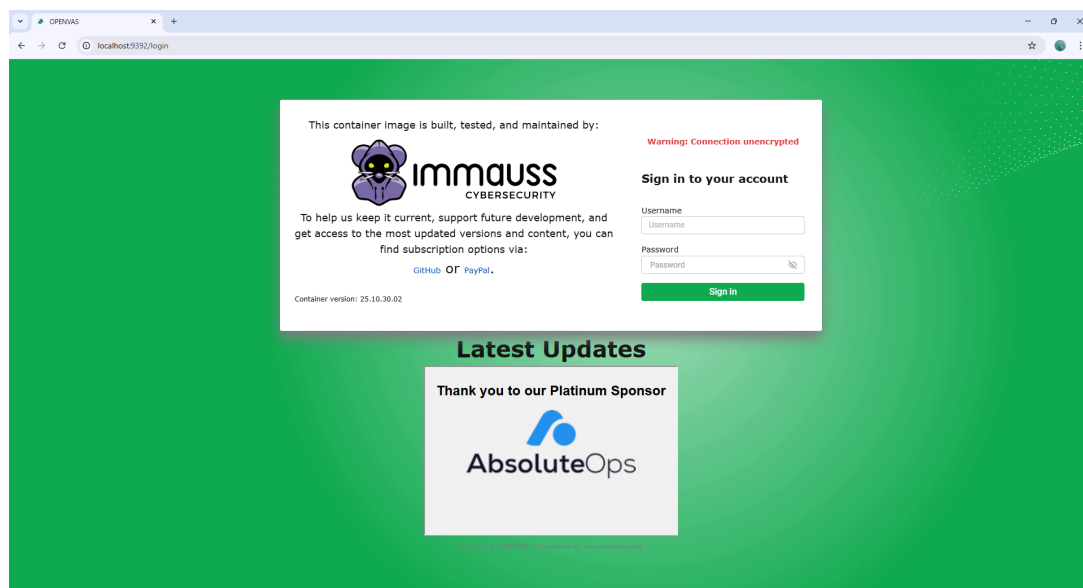


Figure 1: Pantalla de inicio de sesión de OpenVAS con advertencia de conexión sin cifrar

La interfaz de acceso presenta el logo de Immauss Cybersecurity, desarrolladores del contenedor Docker utilizado. Se solicitan credenciales de usuario (Username y Password) para acceder al sistema. La advertencia “Warning: Connection unencrypted” en rojo indica que la conexión HTTP no está cifrada, algo común en instalaciones locales de prueba pero que debería evitarse en entornos de producción mediante el uso de HTTPS. La versión del contenedor es 25.10.30.02, garantizando acceso a las definiciones de vulnerabilidades más recientes.

2.3.4. Panel de Control Principal

Tras iniciar sesión, OpenVAS presenta un dashboard que centraliza toda la información sobre escaneos, vulnerabilidades y activos monitorizados. La Figure 2 muestra esta vista principal.

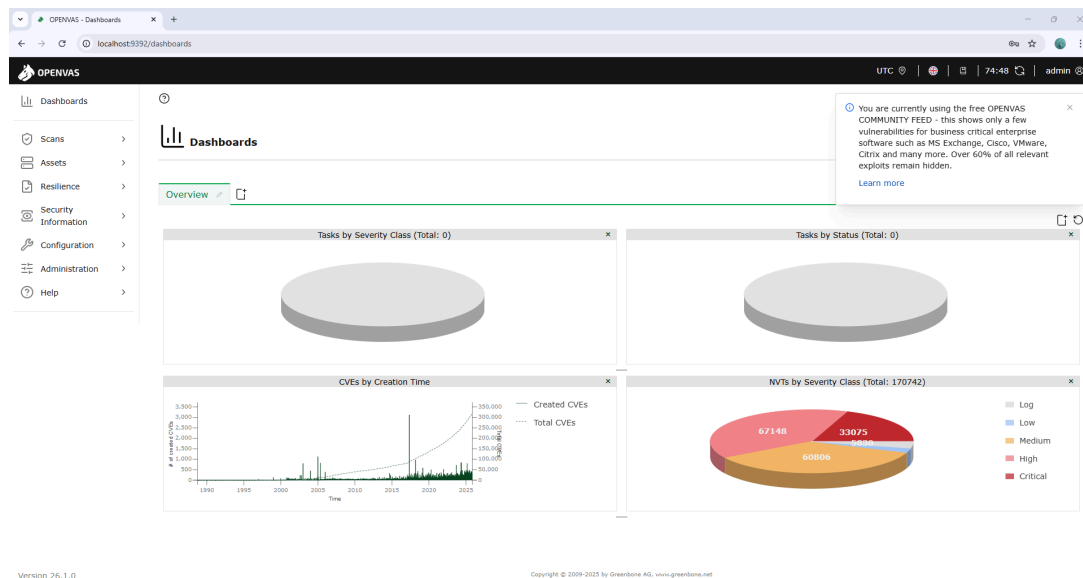


Figure 2: Dashboard principal de OpenVAS mostrando estadísticas de vulnerabilidades y tareas

El panel de control se organiza en varias secciones informativas. En la parte superior derecha aparece un aviso informativo sobre el uso de la versión gratuita (COMMUNITY FEED), que incluye vulnerabilidades críticas para software empresarial como MS Exchange, Cisco y VMware, aunque oculta más del 60% de exploits relevantes disponibles en la versión comercial. Los gráficos circulares superiores muestran “Tasks by Severity Class” y “Tasks by Status”, ambos vacíos inicialmente (Total: 0) ya que aún no se han configurado escaneos.

La sección inferior presenta dos gráficos significativos: “CVEs by Creation Time” muestra la evolución histórica de vulnerabilidades registradas desde 1990 hasta 2025, evidenciando un crecimiento exponencial en los últimos años con más de 300.000 CVEs totales acumulados. El gráfico circular “NVTs by Severity Class” revela la distribución de las 170.742 pruebas de vulnerabilidad disponibles en la base de datos: 67.148 clasificadas como Log, 68.806 como Low, 33.075 como Medium, 1.658 como High y 55 como Critical. Esta distribución muestra que la mayoría de las pruebas detectan problemas informativos o de baja severidad.

El menú lateral izquierdo proporciona acceso a todas las funcionalidades: Dashboards, Scans (con submenús para Tasks, Reports, Results y Vulnerabilities), Assets, Resilience, Security Information, Configuration, Administration y Help.

2.3.5. Creación de Nueva Tarea de Escaneo

Para iniciar un análisis de vulnerabilidades, es necesario crear una tarea que defina qué sistemas analizar y con qué configuración. La Figure 3 muestra el formulario de creación.

Figure 3: Formulario de creación de nueva tarea de escaneo en OpenVAS

El formulario “New Task” permite configurar múltiples parámetros del escaneo. El campo “Name” identifica la tarea (por defecto “Unnamed”), mientras que “Comment” permite añadir descripciones adicionales. La sección “Scan Targets” define qué sistemas serán analizados, seleccionándose de una lista de objetivos previamente configurados. “Alerts” permite configurar notificaciones automáticas cuando se detecten vulnerabilidades críticas.

La configuración de “Schedule” determina si el escaneo se ejecuta una sola vez (Once) o de forma recurrente. “Add results to Assets” con valor “Yes” seleccionado garantiza que los sistemas escaneados se registren automáticamente en el inventario de activos de la organización. “Apply Overrides” con valor “Yes” permite aplicar excepciones personalizadas a vulnerabilidades conocidas que han sido aceptadas o mitigadas.

El parámetro “Min QoD” (Quality of Detection) establecido en 70% filtra los resultados para mostrar solo aquellas vulnerabilidades cuya detección tiene al menos un 70% de fiabilidad, reduciendo así los falsos positivos. “Alterable Task” habilitado permite modificar la configuración de la tarea incluso después de haberla creado. Finalmente, “Auto Delete Reports” con valor “Do not automatically delete reports” conserva todos los informes históricos para análisis comparativos posteriores.

2.3.6. Vista de Tareas Configuradas

Una vez creada la tarea de escaneo, esta aparece listada en el panel de administración de tareas. La Figure 4 muestra la tarea recién configurada.

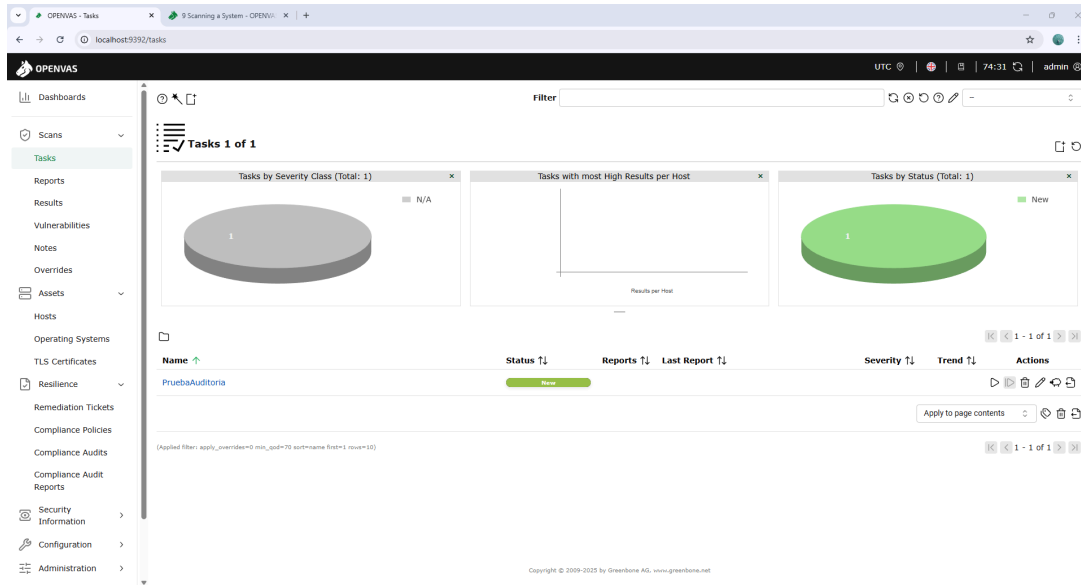


Figure 4: Panel de tareas mostrando la tarea “PruebaAuditoria” lista para ejecutar

La vista de tareas presenta gráficos actualizados en la parte superior: “Tasks by Severity Class” indica “N/A” porque la tarea aún no ha sido ejecutada, mientras que “Tasks by Status” muestra 1 tarea en estado “New” (verde), y “Tasks with most High Results per Host” permanece vacío. La tabla inferior lista la tarea “PruebaAuditoria” con estado “New” destacado en verde, indicando que está lista para ser ejecutada pero aún no ha comenzado. Las columnas “Reports” y “Last Report” están vacías porque no se han generado resultados todavía.

En la columna “Actions” (extremo derecho) aparecen varios iconos que permiten iniciar el escaneo (▶), pausar, detener, editar configuración o eliminar la tarea. Al hacer clic en el icono de reproducción, OpenVAS comenzaría el análisis del sistema objetivo, proceso que típicamente tarda entre 15 y 45 minutos dependiendo de la configuración elegida y la complejidad del objetivo.

2.3.7. Resultados del Escaneo

Tras completar el escaneo, OpenVAS presenta un informe detallado con todas las vulnerabilidades detectadas. La Figure 5 muestra la lista de hallazgos.

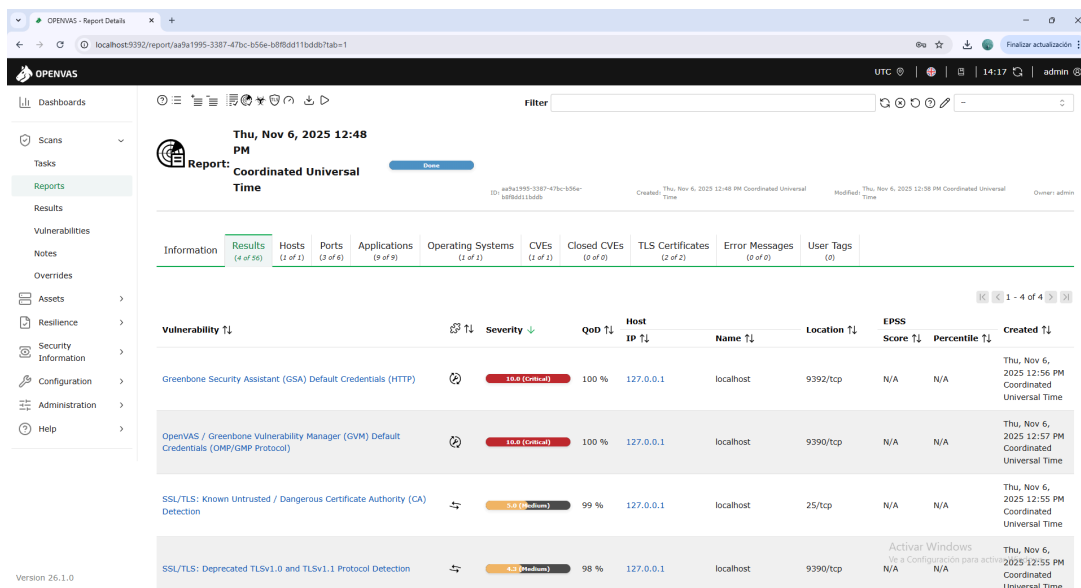


Figure 5: Lista de vulnerabilidades detectadas clasificadas por severidad

El encabezado del informe indica que fue generado el 6 de noviembre de 2025 a las 12:48 PM bajo el esquema de tiempo “Coordinated Universal Time”. El estado “Done” confirma que el escaneo finalizó exitosamente. Las pestañas superiores organizan la información en categorías: Results (4 de 56 mostrados), Hosts (1 de 1), Ports (3 de 6), Applications (5 de 9), Operating Systems (1 de 1), CVEs (1 de 1), Closed CVEs (0 de 0), TLS Certificates (2 de 2), Error Messages (0 de 0) y User Tags (0).

La tabla principal lista las vulnerabilidades detectadas con columnas clave: “Vulnerability” describe el problema encontrado, “Severity” clasifica su criticidad mediante colores (rojo para Critical 10.0, naranja para Medium), “QoD” (Quality of Detection) indica la fiabilidad de la detección (98-100%), “Host” muestra que todos los hallazgos afectan a 127.0.0.1 (localhost), “Location” especifica el puerto afectado, y “Created” registra cuándo se detectó cada vulnerabilidad.

Entre los hallazgos destacados aparecen dos vulnerabilidades críticas de severidad 10.0 (máxima): “Greenbone Security Assistant (GSA) Default Credentials (HTTP)” y “OpenVAS / Greenbone Vulnerability Manager (GVM) Default Credentials (OMP/GMP Protocol)”, ambas con 100% de confianza. Estas indican que el sistema OpenVAS utiliza credenciales por defecto, un riesgo extremo en entornos de producción. También se detectaron vulnerabilidades de severidad media relacionadas con certificados SSL/TLS no confiables o desactualizados en los puertos 25/tcp y 9390/tcp.

2.3.8. Detalle de Vulnerabilidad Específica

Para comprender completamente una vulnerabilidad y cómo remediarla, OpenVAS proporciona información técnica detallada. La Figure 6 muestra el análisis completo de un hallazgo.

The screenshot shows the OpenVAS web interface. The top navigation bar includes 'Dashboards', 'Scans', 'Reports', 'Results', 'Vulnerabilities', 'Hosts', 'Overrides', 'Assets', 'References', 'Security Information', 'Configuration', 'Administration', and 'Help'. The main content area displays a table of vulnerabilities. The selected entry is 'SSL/TLS: Known Untrusted / Dangerous Certificate Authority (CA) Detection'. The report details include:

- Summary:** The service is using an SSL/TLS certificate from a known untrusted and/or dangerous certificate authority (CA).
- Detection Result:** The certificate of the remote service is signed by the following untrusted and/or dangerous CA: Issuer: CN=localhost. Certificate details include fingerprint (SHA-1), fingerprint (SHA-256), issued by, public key algorithm, public key size (bits), serial, signature algorithm, subject, subject's alternative names (SAN), valid from, and valid until.
- Product Detection Result:** Product: gsa/self-transport_secure, Method: SSL/TLS: Collect and Report Certificate Details (GDD: 1.3.6.1.4.1.23623.1.0.10390), Log: View details of product detection.
- Detection Method:** The script reads the certificate used by the target host and checks if it was signed by a known untrusted and/or dangerous CA. Details: SSL/TLS: Known Untrusted / Dangerous Certificate Authority (CA) Detected - GDD: 1.3.6.1.4.1.23623.1.0.10390. Version used: 2024-06-04T05:05:48Z.
- Impact:** An attacker could use this for man-in-the-middle (MITM) attacks, accessing sensitive data and other attacks.
- Solution:** Replace the SSL/TLS certificate with one signed by a trusted CA.

Figure 6: Detalle completo de una vulnerabilidad SSL/TLS con descripción, impacto y solución

El panel de detalle para la vulnerabilidad “SSL/TLS: Known Untrusted / Dangerous Certificate Authority (CA) Detection” proporciona múltiples secciones informativas. El “Summary” explica que el servicio utiliza un certificado SSL/TLS firmado por una autoridad certificadora (CA) conocida como no confiable o peligrosa, con severidad 5.4 (Medium) y 99% de confianza en la detección.

La sección “Detection Result” detalla técnicamente el problema: el certificado del servicio remoto está firmado por una CA no confiable. Se especifica el emisor (CN=localhost), detalles del certificado incluyendo sus huellas digitales (fingerprint SHA-1 y SHA-256), el algoritmo de firma (sha256WithRSAEncryption), el tamaño de clave pública (2048 bits), fechas de validez (2025-10-26 al 2035-10-22) y nombres alternativos del sujeto (CN=localhost).

“Product Detection Result” identifica que se detectó el producto “cpe:/a:ietf:transport_layer_security” mediante el método SSL/TLS con OID específico. El apartado “Detection Method” explica que el script lee el certificado usado por el host objetivo y verifica si fue firmado por una CA conocida como no confiable o peligrosa, proporcionando referencias al OID de detección correspondiente.

La sección “Impact” advierte que un atacante podría usar esta vulnerabilidad para ataques man-in-the-middle (MITM), accediendo a datos sensibles y otros tipos de ataques. Finalmente, “Solution” recomienda como “Mitigation” (mitigación) reemplazar el certificado SSL/TLS con uno firmado por una CA confiable, proporcionando así una guía clara de remediación para los administradores del sistema.

2.3.9. Generación de Informes

OpenVAS permite exportar los resultados del escaneo en múltiples formatos para documentación, cumplimiento normativo o presentación a directivos. La Figure 7 muestra el informe generado en formato PDF.

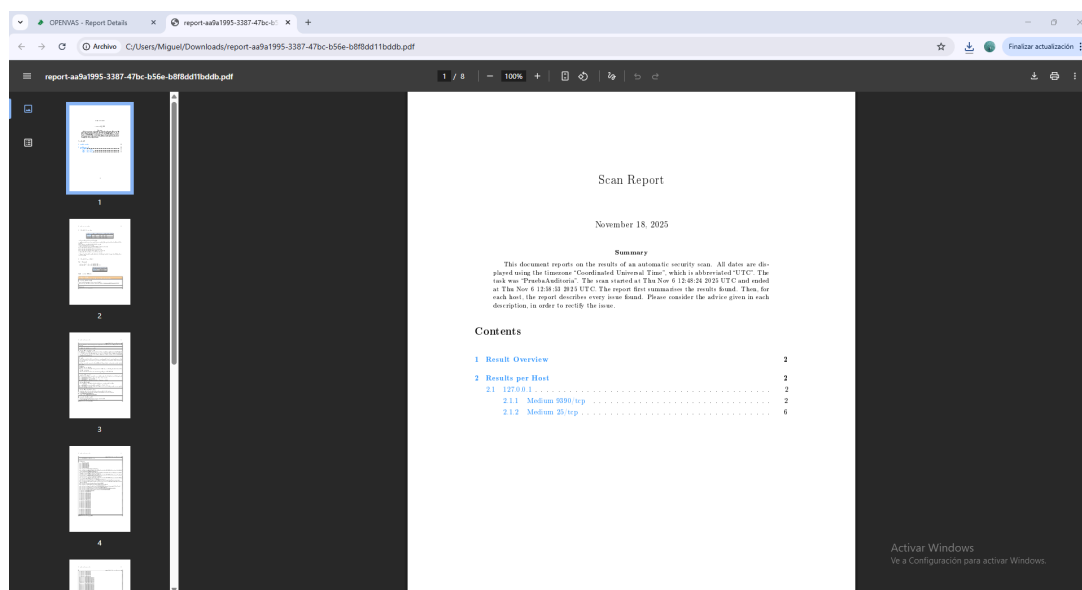


Figure 7: Reporte PDF generado por OpenVAS con resumen ejecutivo y tabla de contenidos

El informe PDF presenta una portada profesional titulada “Scan Report” con la fecha de generación (November 18, 2025). El “Summary” explica que el documento reporta los resultados de un escaneo automático de seguridad realizado con el formato “Coordinated Universal Time” abreviado como “UTC”. Especifica que la tarea “PruebaAuditoria” inició el 6 de noviembre de 2025 a las 12:48:24 UTC y finalizó a las 12:49:01 UTC, con una duración total de aproximadamente 37 segundos.

El resumen advierte que se muestran únicamente datos de un único host (el objetivo del escaneo), y que para cada host encontrado, el informe describe cada hallazgo de seguridad. Concluye recomendando considerar cuidadosamente las soluciones propuestas en cada descripción para rectificar los problemas identificados.

La sección “Contents” organiza el documento en dos apartados principales: “Result Overview” (página 2) que presenta un resumen general de los hallazgos, y “Results per Host” (también página 2) que detalla específicamente los resultados para el host 127.0.0.1, subdividido en vulnerabilidades de severidad “Medium 6590/tcp” (página 2) y “Medium 25/tcp” (página 6). Esta estructura facilita la navegación y permite a los lectores localizar rápidamente la información relevante según su rol o necesidades específicas.

2.3.10. Conclusiones del Caso Práctico

El análisis realizado con OpenVAS sobre el sistema local (127.0.0.1) detectó un total de 56 vulnerabilidades, de las cuales destacan 2 hallazgos críticos de severidad 10.0 relacionados con el uso de credenciales por defecto en los componentes de Greenbone (GSA y GVM). Estas vulnerabilidades representan riesgos extremos que deben remediarse inmediatamente en cualquier entorno de producción, aunque en este caso corresponden al propio sistema OpenVAS instalado en modo de prueba.

Los hallazgos de severidad media se concentraron principalmente en problemas de certificados SSL/TLS no confiables o autofirmados en diversos servicios (puertos 25/tcp, 9390/tcp, 6590/tcp), así como en protocolos TLS desactualizados. Estos problemas, aunque menos críticos, podrían facilitar ataques de intermediario (man-in-the-middle) y deberían abordarse mediante la implementación de certificados firmados por autoridades certificadoras confiables y la desactivación de protocolos obsoletos como TLSv1.0 y TLSv1.1.

OpenVAS demuestra ser una herramienta robusta para la identificación sistemática de vulnerabilidades, proporcionando no solo la detección de problemas sino también información técnica detallada sobre cada hallazgo, su impacto potencial y recomendaciones concretas de remediación. La capacidad de generar informes profesionales en múltiples formatos facilita la comunicación de los resultados tanto a equipos técnicos como a directivos no especializados.

La base de datos de 170.742 pruebas de vulnerabilidad (NVTs) garantiza una cobertura exhaustiva de problemas de seguridad conocidos, aunque la versión gratuita oculta aproximadamente el 60% de exploits relevantes para software empresarial crítico. Para entornos corporativos de producción, sería recomendable considerar la versión comercial que proporciona acceso completo a todas las definiciones de vulnerabilidades.

En conclusión, OpenVAS se posiciona como una herramienta esencial en el arsenal de cualquier profesional de seguridad informática, permitiendo identificar proactivamente debilidades antes de que sean explotadas por atacantes, cumplir con requisitos de auditoría y normativas de seguridad, y mantener una postura de seguridad sólida mediante escaneos periódicos automatizados.

3. Wireshark

En el ámbito de la seguridad informática y la administración de redes, el análisis del tráfico de datos es una actividad fundamental para garantizar el correcto funcionamiento y la protección de la infraestructura tecnológica. En este contexto, Wireshark [1] se consolida como una de las herramientas más reconocidas y potentes para la captura y análisis de paquetes de red, permitiendo a administradores, analistas de seguridad y técnicos comprender en detalle lo que sucede dentro de una red de comunicaciones.

Wireshark es una herramienta de código abierto diseñada para examinar el tráfico de red en tiempo real o a partir de capturas almacenadas, ofreciendo una visión profunda del intercambio de datos entre dispositivos. Permite inspeccionar protocolos, identificar anomalías, diagnosticar problemas de rendimiento y detectar posibles amenazas de seguridad. Se clasifica dentro de las herramientas de: [2]

- Análisis y monitoreo de tráfico de red.
- Auditoría y diagnóstico de comunicaciones.
- Seguridad informática y detección de intrusiones.

3.1. Características Principales

La herramienta **Wireshark** destaca por las siguientes funcionalidades clave:

1. **Código abierto y gratuito:** Disponible bajo licencia GNU GPL, lo que facilita su uso y adaptación.
2. **Compatibilidad multiplataforma:** Funciona en sistemas **Windows, Linux, macOS** y **Unix**.
3. **Captura en tiempo real:** Permite observar el tráfico de red conforme se produce, mostrando detalles de cada paquete.
4. **Amplio soporte de protocolos:** Reconoce y analiza más de **2.000 protocolos** de red, desde los más comunes (**TCP/IP, HTTP, DNS**) hasta los más específicos.
5. **Filtros avanzados:** Ofrece potentes opciones de filtrado para aislar tráfico relevante durante el análisis.
6. **Interfaz gráfica intuitiva:** Proporciona una vista detallada y estructurada de las tramas capturadas, con decodificación jerárquica.
7. **Exportación de datos:** Permite guardar capturas en múltiples formatos y generar reportes para auditorías o investigaciones.
8. **Integración con otras herramientas:** Compatible con utilidades como **Tshark** (versión de línea de comandos) o **NetworkMiner** para análisis complementario.

3.2. Ventajas y Desventajas

Entre sus ventajas, Wireshark sobresale por su potencia, precisión y gratuidad, convirtiéndose en un estándar de facto para el análisis de tráfico en redes empresariales y académicas. Su capacidad para desglosar paquetes a nivel de bits facilita la detección de errores de configuración, vulnerabilidades o comportamientos anómalos en la red. Además, su comunidad activa y abundante documentación favorecen su aprendizaje y soporte continuo.

No obstante, presenta ciertos **inconvenientes**: requiere conocimientos técnicos avanzados en redes y protocolos para interpretar correctamente los datos capturados. Asimismo, en entornos grandes o con tráfico cifrado, su uso puede volverse más complejo y demandar un mayor consumo de recursos del sistema. Aun así, Wireshark sigue siendo una herramienta imprescindible para la **auditoría, diagnóstico y seguridad de redes**, combinando potencia, flexibilidad y un entorno completamente abierto.

3.3. Wireshark - Caso Práctico

3.3.1. Descripción del Caso

El objetivo de este caso práctico es analizar el tráfico de red generado durante una sesión de navegación web utilizando Wireshark. Se capturará y examinará el tráfico a nivel de paquetes para identificar los principales protocolos de comunicación y comprender cómo las aplicaciones intercambian información a través de la red.

3.3.2. Entorno de Pruebas

- **Herramienta:** Wireshark 4.2.2
- **Sistema Operativo:** Linux 6.14.0-35-generic (Ubuntu)
- **Duración de captura:** 15 minutos 57 segundos
- **Paquetes capturados:** 21.408 paquetes
- **Tamaño de captura:** 59 MB

3.3.3. Captura y Visualización del Tráfico

Al iniciar Wireshark y comenzar la captura, la herramienta intercepta todos los paquetes que circulan por la red. La Figure 8 muestra la interfaz principal durante una sesión de captura activa.

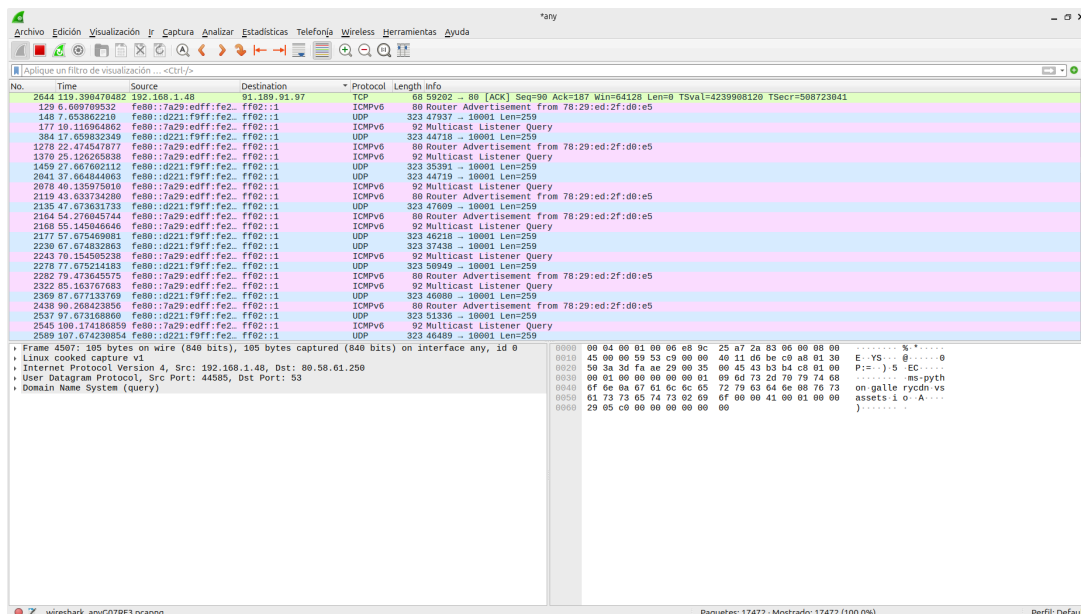


Figure 8: Interfaz principal de Wireshark mostrando la captura de tráfico de red en tiempo real

La interfaz se divide en tres paneles: el superior muestra la lista de paquetes capturados con información básica como origen, destino y protocolo; el panel central presenta los detalles del paquete seleccionado en estructura jerárquica; y el panel inferior muestra los datos en formato hexadecimal y ASCII. En esta captura se observa principalmente tráfico TCP, UDP e ICMPv6, con más de 17.000 paquetes capturados durante la sesión.

3.3.4. Estructura de un Paquete de Red

Para comprender cómo viaja la información por la red, es fundamental examinar la estructura interna de un paquete. La Figure 9 muestra el desglose completo de un paquete HTTP.

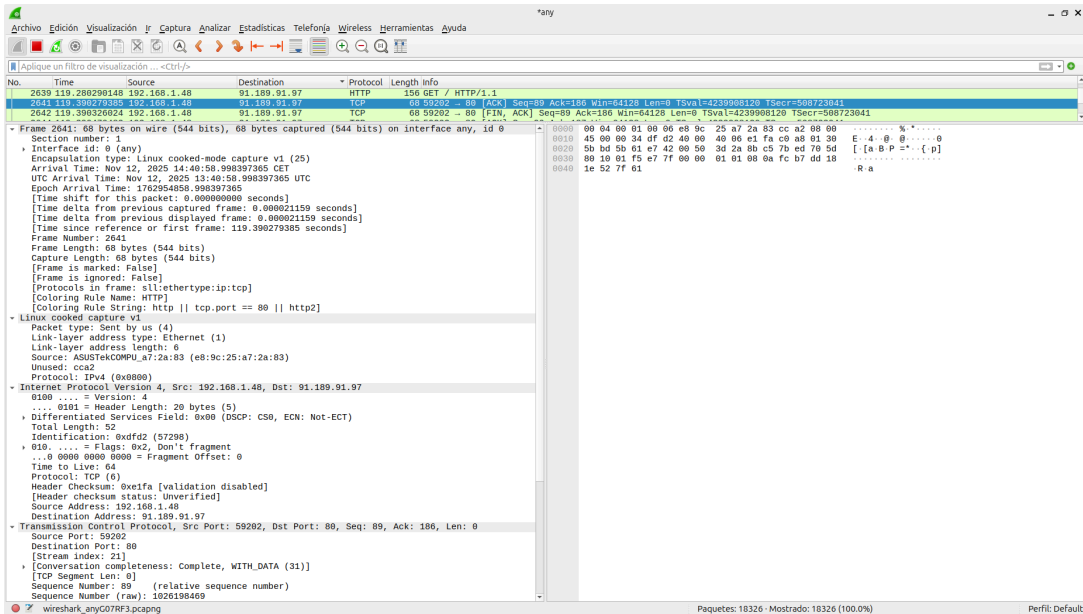


Figure 9: Desglose detallado de un paquete HTTP mostrando todas sus capas

Al expandir un paquete, Wireshark revela su estructura en capas: la capa de Frame contiene información sobre la captura física; la capa IP muestra las direcciones de origen (192.168.1.48) y destino (91.189.91.97); la capa TCP indica los puertos de comunicación (origen 59202, destino 80); y finalmente, la capa HTTP transporta la petición web propiamente dicha. Esta organización en capas ejemplifica el modelo TCP/IP, donde cada nivel añade su propia información antes de transmitir los datos.

3.3.5. Análisis de Protocolo DNS

El DNS (Sistema de Nombres de Dominio) traduce nombres de dominio legibles como “google.com” en direcciones IP que las computadoras pueden usar. La Figure 10 muestra el tráfico DNS aislado mediante un filtro.

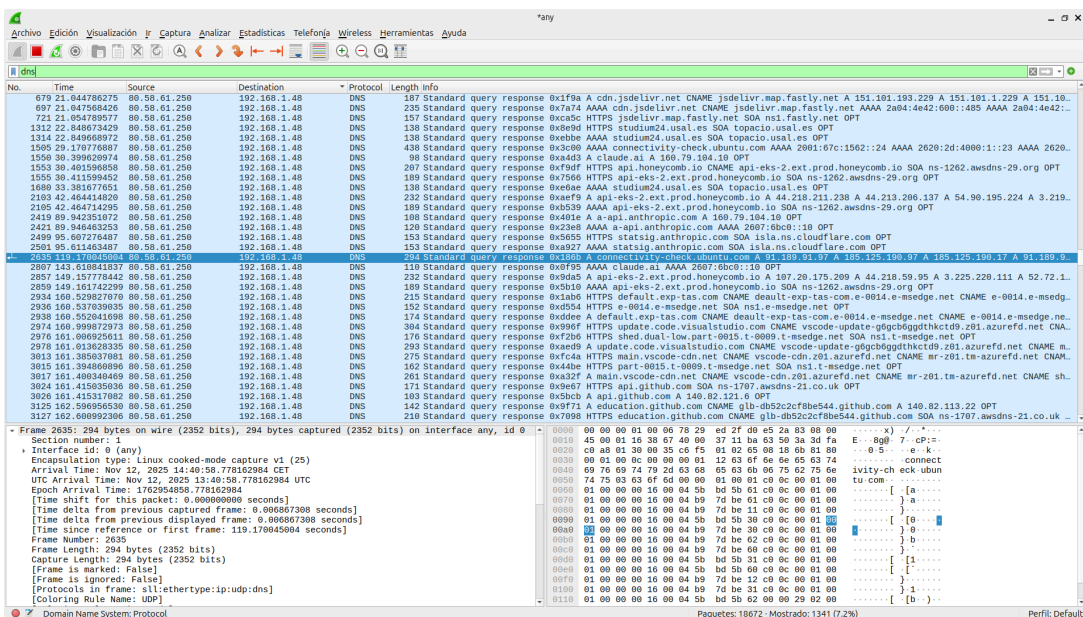


Figure 10: Tráfico DNS filtrado mostrando consultas y respuestas de resolución de nombres

Al aplicar el filtro dns, Wireshark muestra únicamente los paquetes relacionados con resoluciones de nombres de dominio. Se observan consultas del equipo local (192.168.1.48) al servidor DNS

(80.58.61.250) preguntando por diversos dominios como studium25.usal.es, claude.ai, github.com, entre otros. Cada consulta genera una respuesta que contiene la dirección IP correspondiente al dominio solicitado. Este tráfico DNS representa el 7.2% del total capturado, evidenciando la navegación activa por múltiples sitios web.

3.3.6. Detalle de Resolución DNS

Para entender mejor cómo funciona una consulta DNS, la Figure 11 muestra la estructura completa de una respuesta DNS.

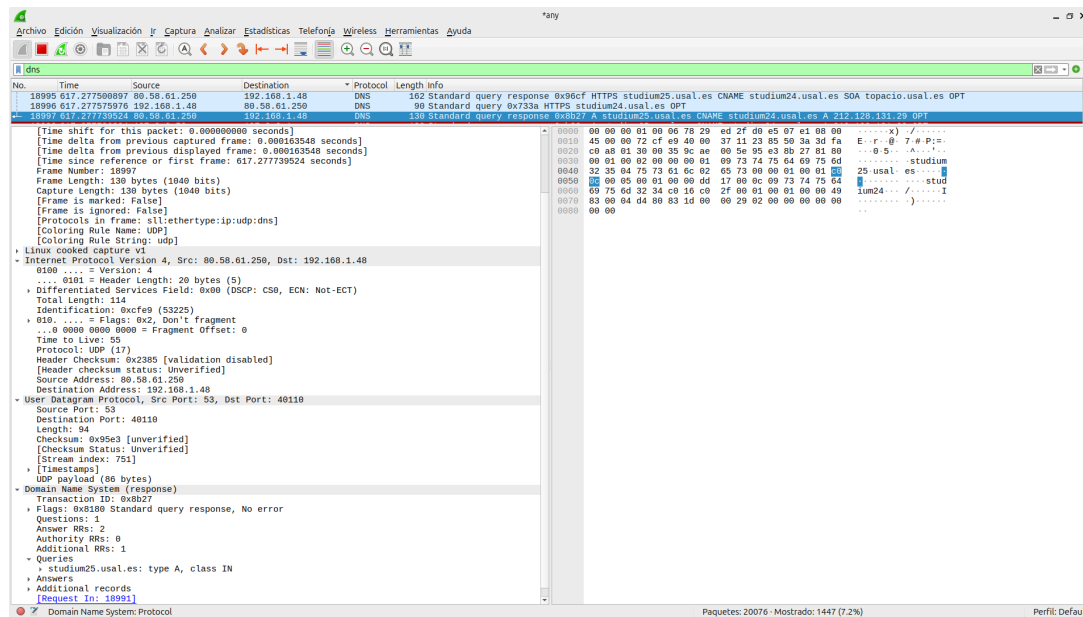


Figure 11: Detalle de una respuesta DNS mostrando la resolución de studium25.usal.es

La respuesta DNS contiene varios elementos clave: primero, repite la consulta original (studium25.usal.es); luego proporciona la respuesta con la dirección IP resuelta (212.128.131.29); y finalmente incluye información adicional como el tiempo de vida (TTL) que indica cuánto tiempo es válida esta respuesta antes de necesitar una nueva consulta. El protocolo DNS utiliza UDP por su rapidez, ya que no requiere establecer una conexión completa como TCP.

3.3.7. Análisis de Tráfico HTTP

HTTP es el protocolo fundamental de la web que permite la transferencia de páginas, imágenes y otros recursos. La Figure 12 muestra el tráfico HTTP sin cifrar capturado.

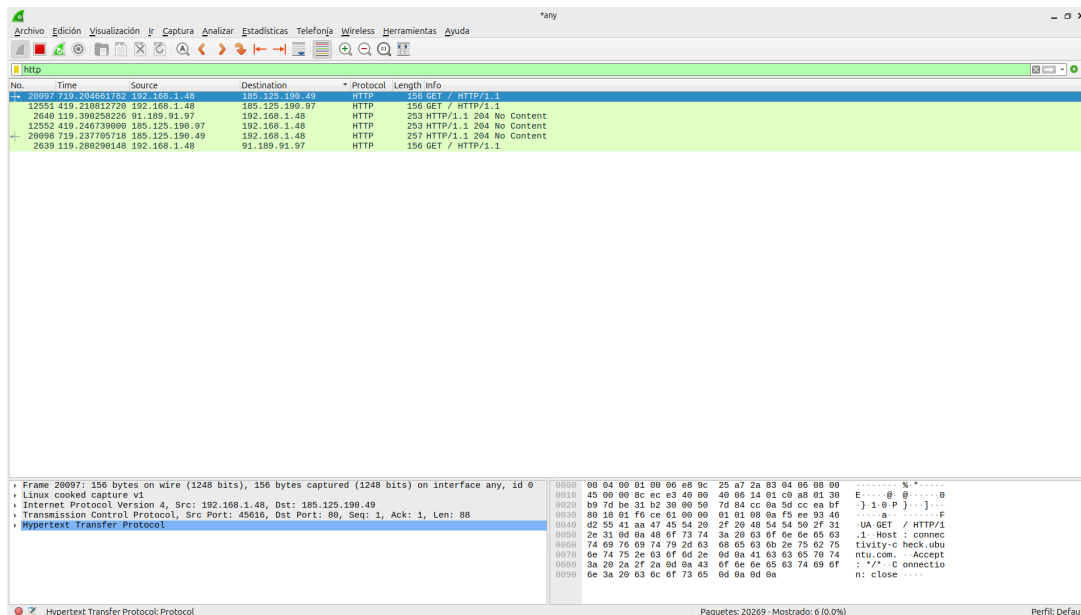


Figure 12: Tráfico HTTP filtrado mostrando peticiones y respuestas web

El filtro `http` revela solo 6 paquetes de los más de 20.000 capturados, representando el 0.0% del tráfico total. Esta mínima presencia de HTTP sin cifrar refleja que la mayoría del tráfico web actual utiliza HTTPS (HTTP con cifrado TLS), mucho más seguro. Los pocos paquetes HTTP observados corresponden principalmente a verificaciones automáticas de conectividad del sistema operativo Ubuntu, que utiliza peticiones HTTP simples para comprobar si hay acceso a Internet. Las respuestas muestran códigos “204 No Content”, indicando que la conexión fue exitosa sin necesidad de devolver contenido.

3.3.8. Estructura de una Petición HTTP

La Figure 13 presenta el detalle completo de una petición HTTP, revelando toda la información que el navegador envía al servidor.

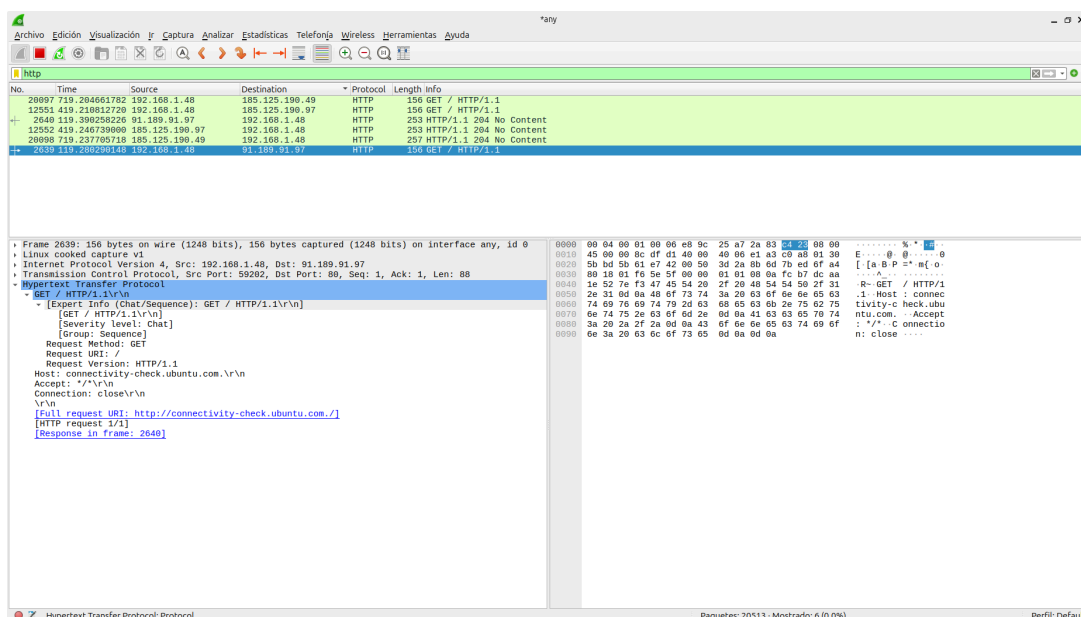


Figure 13: Detalle de una petición HTTP GET mostrando todos sus componentes

Una petición HTTP comienza con la línea de petición que especifica el método (GET), el recurso solicitado (/) y la versión del protocolo (HTTP/1.1). A continuación aparecen los headers: “Host” indica el

servidor destino (connectivity-check.ubuntu.com), “Accept” especifica qué tipos de contenido acepta el cliente, y “Connection: close” solicita cerrar la conexión tras recibir la respuesta. Esta petición específica forma parte del mecanismo automático de Ubuntu para verificar la conectividad a Internet. El panel hexadecimal muestra estos mismos datos en formato binario, tal como viajan por la red.

3.3.9. Reconstrucción de Conversaciones TCP

Una funcionalidad muy útil de Wireshark es la capacidad de reconstruir conversaciones completas entre cliente y servidor. La Figure 14 muestra una sesión HTTP completa.

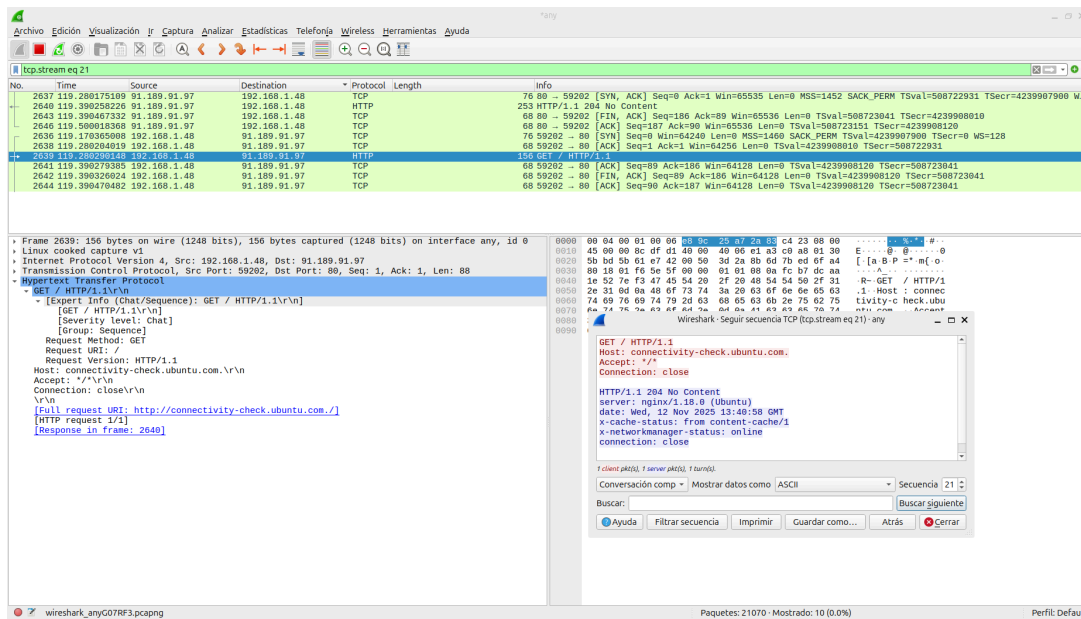


Figure 14: Reconstrucción de una conversación HTTP completa entre cliente y servidor

Al seleccionar “Follow TCP Stream” sobre un paquete, Wireshark reconstruye toda la conversación de esa conexión TCP, mostrando tanto la petición del cliente como la respuesta del servidor. En la ventana emergente se observa claramente la petición HTTP GET en la parte superior, seguida de la respuesta del servidor en la parte inferior. La respuesta incluye el código de estado (HTTP/1.1 204 No Content), información del servidor (nginx/1.18.0) y un header especial (x-networkmanager-status: online) que confirma la conectividad. Esta vista unificada facilita enormemente el análisis de protocolos basados en texto como HTTP.

3.3.10. Estadísticas de la Captura

Wireshark proporciona información detallada sobre las características de cada captura realizada. La Figure 15 muestra las propiedades completas del archivo.

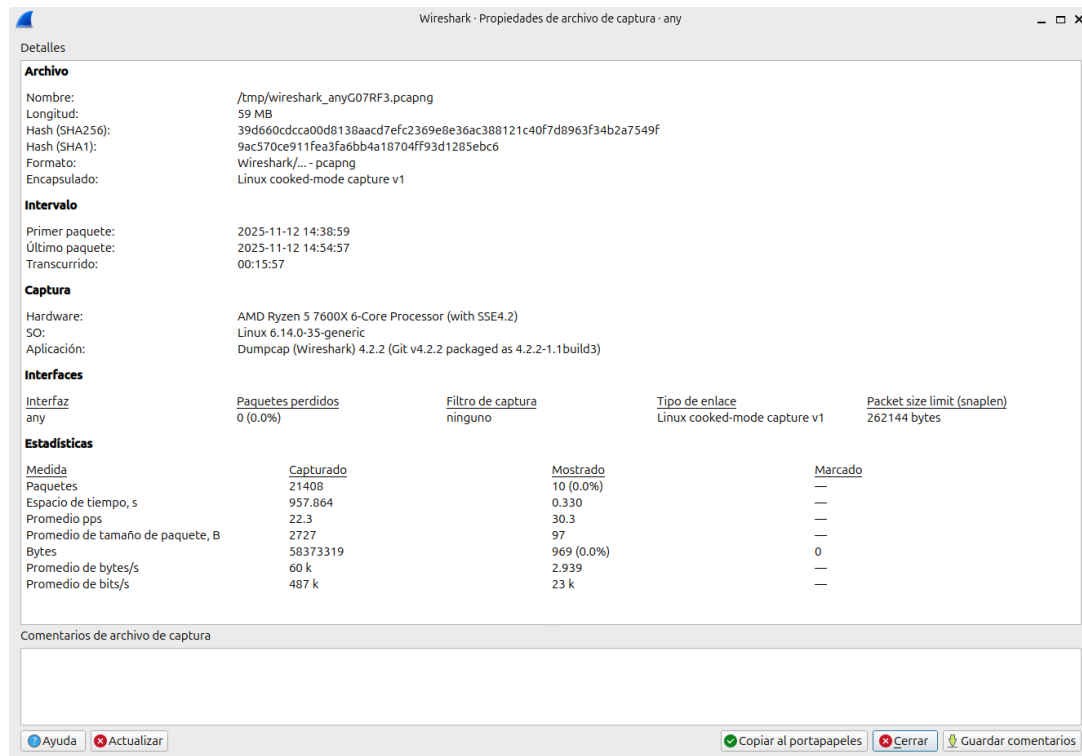


Figure 15: Propiedades y estadísticas completas del archivo de captura

Las estadísticas revelan que la captura abarcó 15 minutos y 57 segundos, durante los cuales se capturaron 21.408 paquetes totalizando 59 MB. El sistema capturó en la interfaz “any”, lo que significa que se monitorearon todas las interfaces de red simultáneamente. Un dato importante es que no se perdió ningún paquete (0.0% de pérdida), garantizando la integridad del análisis. El throughput promedio fue de 487 kbps, con 22.3 paquetes por segundo, valores típicos de una sesión de navegación web normal sin descargas pesadas. El formato de archivo es pcapng, un estándar moderno que permite almacenar metadata adicional junto con los paquetes capturados.

3.3.11. Conclusiones del Caso Práctico

El análisis realizado con Wireshark ha permitido observar directamente cómo funciona la comunicación en redes de computadoras. Se identificaron los protocolos más comunes en la navegación web actual: DNS para resolver nombres de dominio, TCP para establecer conexiones confiables, y HTTP/HTTPS para transferir contenido web.

Los hallazgos principales incluyen la predominancia casi absoluta de tráfico cifrado (HTTPS sobre TLS), evidenciado por la mínima presencia de HTTP sin cifrar. El alto volumen de consultas DNS refleja la naturaleza distribuida de la web moderna, donde una simple página web puede requerir recursos de docenas de dominios diferentes. La estructura en capas observada en cada paquete ejemplifica el modelo TCP/IP, fundamental para entender cómo la información se encapsula y transmite a través de múltiples niveles de protocolos.

Wireshark demuestra ser una herramienta invaluable para la auditoría y diagnóstico de redes, permitiendo visualizar y analizar el tráfico de red con un nivel de detalle excepcional. Su capacidad para filtrar, reconstruir conversaciones y presentar estadísticas la convierte en una herramienta esencial para administradores de sistemas, profesionales de seguridad y cualquier persona interesada en comprender el funcionamiento interno de las comunicaciones de red.

4. Lynis

Lynis [3] se presenta como una solución especializada de código abierto para la auditoría de seguridad en sistemas Unix y Linux, proporcionando a los administradores una visión integral del estado de seguridad del sistema operativo y sus componentes.

Lynis es una herramienta de auditoría de seguridad diseñada específicamente para sistemas basados en Unix, que realiza análisis profundos de la configuración del sistema, identifica vulnerabilidades potenciales y proporciona recomendaciones concretas para mejorar la postura de seguridad. A diferencia de las herramientas centradas en el análisis de red, Lynis se enfoca en el sistema operativo local, evaluando aspectos como la configuración de servicios, políticas de contraseñas, permisos de archivos y cumplimiento de estándares de seguridad.

Se clasifica dentro de las herramientas de: [4]

- Auditoría de seguridad de sistemas operativos.
- Evaluación de hardening y configuración de servidores.
- Análisis de cumplimiento normativo y mejores prácticas.

4.1. Características Principales

La herramienta **Lynis** destaca por las siguientes funcionalidades clave:

1. **Código abierto y multiplataforma:** Disponible bajo licencia GPL, compatible con **Linux, Unix, macOS, BSD** y otros sistemas Unix-like.
2. **Auditoría exhaustiva del sistema:** Examina más de 250 aspectos de seguridad incluyendo kernel, servicios, usuarios, archivos y configuraciones.
3. **Índice de endurecimiento (Hardening Index):** Proporciona una puntuación numérica que refleja el nivel de seguridad del sistema, facilitando la medición de mejoras.
4. **Detección de vulnerabilidades:** Identifica configuraciones inseguras, permisos incorrectos, servicios innecesarios y actualizaciones pendientes.
5. **Análisis sin agentes:** No requiere instalación permanente ni servicios en segundo plano, puede ejecutarse desde un medio extraíble.
6. **Generación de informes detallados:** Crea registros completos en `/var/log/lynis.log` y `/var/log/lynis-report.dat` con hallazgos y recomendaciones específicas.
7. **Categorización de resultados:** Clasifica los hallazgos en sugerencias, advertencias y notas, priorizando las acciones correctivas.
8. **Integración en sistemas de auditoría:** Compatible con herramientas de automatización y sistemas de gestión de configuración como **Ansible, Puppet** o **Chef**.

4.2. Ventajas y Desventajas

Entre sus ventajas, Lynis destaca por su especialización en auditoría de sistemas operativos, ofreciendo una cobertura integral que abarca desde la configuración del kernel hasta las políticas de usuarios y permisos. Su capacidad para generar un índice de endurecimiento cuantificable permite a las organizaciones medir objetivamente su progreso en la implementación de controles de seguridad. Además, al ser una herramienta ligera y no intrusiva, puede ejecutarse regularmente sin afectar el rendimiento del sistema, lo que la convierte en ideal para auditorías periódicas y automatizadas.

No obstante, presenta ciertas **limitaciones**: está principalmente diseñada para sistemas Unix/Linux, dejando fuera entornos Windows nativos. Su efectividad depende de la interpretación correcta de los resultados por parte del administrador, requiriendo conocimientos sólidos en administración de sistemas y seguridad. Asimismo, aunque identifica problemas y proporciona recomendaciones, no realiza correcciones automáticas, delegando la implementación de mejoras al usuario.

4.3. Lynis - Caso Práctico

4.3.1. Descripción del Caso

El presente caso práctico tiene como objetivo realizar una auditoría completa de seguridad sobre un sistema **Ubuntu 24.04 LTS** utilizando Lynis. Se evaluará el estado de seguridad del sistema operativo, identificando configuraciones inseguras, vulnerabilidades potenciales y oportunidades de mejora en el hardening del sistema.

4.3.2. Entorno de Pruebas

- **Sistema Operativo:** Ubuntu 24.04.2 LTS (Noble)
- **Kernel:** Linux 6.14.0
- **Herramienta:** Lynis 3.0.9
- **Plataforma:** x86_64
- **Usuario:** Ejecución con privilegios de superusuario (sudo)

4.3.3. Instalación y Verificación

El primer paso consiste en instalar Lynis desde los repositorios oficiales de Ubuntu. La instalación se realizó mediante el gestor de paquetes APT, tal como se muestra en la Figure 16.

```
(base) miguel@miguel-System-Product-Name:~$ sudo apt install lynis
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
lynis ya está en su versión más reciente (3.0.9-1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 242 no actualizados.
(base) miguel@miguel-System-Product-Name:~$ lynis show version
3.0.9
(base) miguel@miguel-System-Product-Name:~$
```

Figure 16: Instalación y verificación de la versión de Lynis instalada en el sistema

Como se observa en la captura, el sistema confirmó que Lynis ya se encontraba instalado en su versión más reciente (3.0.9-1), y se verificó correctamente la versión mediante el comando `lynis show version`, obteniendo la versión **3.0.9**.

4.3.4. Ejecución de la Auditoría

Una vez verificada la instalación, se procedió a ejecutar el análisis completo del sistema mediante el comando `sudo lynis audit system`. Este comando inicia un escaneo exhaustivo que examina múltiples aspectos de la configuración de seguridad del sistema operativo.

4.3.4.1. Inicialización del Escaneo

La Figure 17 muestra la fase de inicialización del análisis, donde Lynis detecta automáticamente las características del sistema y carga los perfiles de auditoría correspondientes.

```
miguel@miguel-System-Product-Name: ~$ sudo lynis audit system
[ Lynis 3.0.9 ]

=====
Lynis comes with ABSOLUTELY NO WARRANTY. This is free software, and you are
welcome to redistribute it under the terms of the GNU General Public License.
See the LICENSE file for details about using this software.

2007-2021, CISOFY - https://cisofy.com/lynis/
Enterprise support available (compliance, plugins, interface and tools)
=====

[+] Initializing program
-----
- Detecting OS... [ DONE ]
- Checking profiles... [ DONE ]
- Detecting language and localization [ es ]
-----

Program version: 3.0.9
Operating system: Linux
Operating system name: Ubuntu
Operating system version: 24.04
Kernel version: 6.14.0
Hardware platform: x86_64
Hostname: miguel-System-Product-Name
-----
Profiles: /etc/lynis/default-prf
Log file: /var/log/lynis.log
Report file: /var/log/lynis-report.dat
Report version: 1.0
Plugin directory: /etc/lynis/plugins
-----
Auditors: [ Not Specified ]
Language: es
Test category: all
Test group: all
-----
- Program update status... [ SIN ACTUALIZACIÓN ]

[+] Herramientas del sistema
-----
- Scanning available tools...
- Checking system binaries...

[+] Plugins (Fase 1)
-----
Nota: los plugins contienen pruebas más extensivas y toman más tiempo
- Plugin: debian [
[+] Debian Tests
-----
Check for updates (this was verified by Debian Tests)
```

Figure 17: Fase de inicialización del análisis de Lynis con detección del sistema operativo

Durante esta fase, Lynis identificó correctamente:

- Sistema operativo: **Ubuntu 24.04**
- Versión del kernel: **6.14.0**
- Plataforma hardware: **x86_64**
- Archivos de registro: **/var/log/lynis.log** y **/var/log/lynis-report.dat**
- Estado de actualización del programa: **SIN ACTUALIZACIÓN** disponible

4.3.4.2. Análisis del Kernel y Hardening

La sección de análisis del kernel examina aspectos críticos relacionados con la seguridad a nivel del núcleo del sistema operativo, tal como se aprecia en la Figure 18.

```
[+] Kernel
-----
- Checking default run level [ RUNLEVEL 5 ]
- Checking CPU support (NX/PAE) [ ENCONTRADO ]
  CPU support: PAE and/or NoeXecute supported
- Checking kernel version and release [ HECHO ]
- Checking kernel type [ HECHO ]
- Checking loaded kernel modules [ HECHO ]
  Found 104 active modules
- Checking Linux kernel configuration file [ ENCONTRADO ]
- Checking default I/O kernel scheduler [ NO ENCONTRADO ]
- Checking for available kernel update [ OK ]
- Checking core dumps configuration
  - configuration in systemd conf files [ POR DEFECTO ]
  - configuration in /etc/profile [ POR DEFECTO ]
  - 'hard' configuration in /etc/security/limits.conf [ POR DEFECTO ]
  - 'soft' configuration in /etc/security/limits.conf [ POR DEFECTO ]
- Checking setuid core dumps configuration [ PROTEGIDO ]
- Check if reboot is needed [ NO ]
```

Figure 18: Análisis del kernel y configuración de hardening del sistema

Los resultados de esta sección revelan:

- **Run level:** RUNLEVEL 5 (modo gráfico multiusuario)
- **Soporte de CPU:** PAE y/o NX/Execute Bit soportado correctamente

- **Módulos del kernel:** Se detectaron 194 módulos activos cargados
- **Configuración de hardening:** Se encontraron archivos de configuración en `/etc/sysctl.conf` y `/etc/security/limits.conf`, aunque algunos presentan configuraciones **POR DEFECTO** que podrían mejorarse
- **Core dumps:** Configuración de dumps del sistema detectada con valores por defecto

4.3.4.3. Auditoría de Usuarios y Autenticación

El análisis de usuarios, grupos y mecanismos de autenticación es fundamental para evaluar la seguridad de acceso al sistema. La Figure 19 presenta los hallazgos en esta área crítica.

[+] Usuarios, grupos y autenticación	
- Administrator accounts	[OK]
- Unique UIDs	[OK]
- Consistency of group files (grpck)	[OK]
- Unique group IDs	[OK]
- Unique group names	[OK]
- Password file consistency	[OK]
- Password hashing methods	[SUGERENCIA]
- Checking password hashing rounds	[DESHABILITADO]
- Query system users (non daemons)	[HECHO]
- NIS+ authentication support	[NO HABILITADO]
- NIS authentication support	[NO HABILITADO]
- Sudoers file(s)	[ENCONTRADO]
- Permissions for directory: /etc/sudoers.d	[PELIGRO]
- Permissions for: /etc/sudoers	[OK]
- Permissions for: /etc/sudoers.d/ospd-openvas	[OK]
- Permissions for: /etc/sudoers.d/README	[OK]
- PAM password strength tools	[OK]
- PAM configuration files (pam.conf)	[ENCONTRADO]
- PAM configuration files (pam.d)	[ENCONTRADO]
- PAM modules	[ENCONTRADO]
- LDAP module in PAM	[NO ENCONTRADO]
- Accounts without expire date	[SUGERENCIA]
- Accounts without password	[OK]
- Locked accounts	[OK]
- Checking user password aging (minimum)	[DESHABILITADO]
- User password aging (maximum)	[DESHABILITADO]
- Checking expired passwords	[OK]
- Checking Linux single user mode authentication	[OK]
- Determining default umask	
- umask (/etc/profile)	[NO ENCONTRADO]
- umask (/etc/login.defs)	[SUGERENCIA]
- LDAP authentication support	[NO HABILITADO]
- Logging failed login attempts	[HABILITADO]

Figure 19: Análisis de usuarios, grupos y configuración de autenticación del sistema

Los aspectos evaluados incluyen:

- **Cuentas de administrador:** Identificadas correctamente con estado **OK**
- **Consistencia de archivos:** Los archivos `/etc/passwd`, `/etc/group` y `/etc/shadow` presentan consistencia correcta
- **Métodos de hash de contraseñas:** Se detectó una **SUGERENCIA** para mejorar las rondas de hashing de contraseñas
- **Soporte NIS+:** **NO HABILITADO** (comportamiento esperado en sistemas modernos)
- **Configuración de sudoers:** Alerta de **PELIGRO** para el acceso al archivo `sudoers.d` el resto de archivos tiene permisos correctos.

- **PAM (Pluggable Authentication Modules):** Módulos encontrados pero algunos archivos de configuración **NO ENCONTRADOS**
- **Cuentas sin expiración:** Se detectaron cuentas sin fecha de expiración configurada (**SUGERENCIA**)
- **Umask predeterminado:** Configuración en /etc/profile y /etc/login.defs con una **SUGERENCIA** de mejora

Destaca el hallazgo de que el **envejecimiento de contraseñas** tanto mínimo como máximo está **DESHABILITADO**, lo que representa una oportunidad de mejora significativa en la política de contraseñas del sistema.

4.3.4.4. Análisis de Sistemas de Ficheros

La evaluación de los sistemas de archivos examina la configuración de montaje, particiones y opciones de seguridad asociadas. La Figure 20 muestra los resultados de esta auditoría.

```
[+] Sistemas de ficheros
-----
- Checking mount points
  - Checking /home mount point [ SUGERENCIA ]
  - Checking /tmp mount point [ SUGERENCIA ]
  - Checking /var mount point [ SUGERENCIA ]
- Query swap partitions (fstab) [ OK ]
- Testing swap partitions [ OK ]
- Testing /proc mount (hidepid) [ SUGERENCIA ]
- Checking for old files in /tmp [ OK ]
- Checking /tmp sticky bit [ OK ]
- Checking /var/tmp sticky bit [ OK ]
- ACL support root file system [ HABILITADO ]
- Mount options of / [ OK ]
- Mount options of /dev [ PARCIALMENTE BASTIONADO ]
- Mount options of /dev/shm [ PARCIALMENTE BASTIONADO ]
- Mount options of /run [ BASTIONADO ]
- Total without nodev:7 noexec:27 nosuid:21 ro or noexec (W^X): 11 of total 44
- Disable kernel support of some filesystems
```

Figure 20: Análisis de sistemas de ficheros y configuración de puntos de montaje

Los hallazgos principales en esta sección son:

- **Puntos de montaje críticos:** Se generaron **SUGERENCIAS** para los directorios /home, /tmp y /var, indicando que deberían montarse con opciones de seguridad adicionales
- **Particiones swap:** Detectadas y configuradas correctamente (**OK**)
- **Bit sticky:** Configurado adecuadamente en /tmp y /var/tmp (**OK**)
- **Soporte ACL:** **HABILITADO** en el sistema de archivos raíz
- **Opciones de montaje:**
 - /dev: **PARCIALMENTE BASTIONADO** (algunas opciones de seguridad aplicadas)
 - /dev/shm: **PARCIALMENTE BASTIONADO**
 - /run: **BASTIONADO** (configuración segura completa)
- **Tipos de sistemas de archivos:** Se detectaron 11 de 44 tipos sin soporte noexec, lo que podría representar un riesgo de seguridad menor

4.3.4.5. Warnings y Sugerencias Detectadas

Al finalizar el escaneo, Lynis presenta un resumen consolidado de todos los hallazgos, clasificados en warnings (advertencias) y suggestions (sugerencias). La Figure 21 muestra esta información crítica.

```

=====
- [ Lynis 3.0.9 Results ] -
=====
Warnings (3):
=====
! Found one or more vulnerable packages. [PKGS-7392]
  https://cisofy.com/lynis/controls/PKGS-7392/

! iptables module(s) loaded, but no rules active [FIRE-4512]
  https://cisofy.com/lynis/controls/FIRE-4512/

! Redis configuration file /etc/redis/redis-openvas.conf is world readable and might leak sensitive details [DBS-1882]
  Details : /etc/redis/redis-openvas.conf
  Solution : Use chown 640 to change file permissions
  https://cisofy.com/lynis/controls/DBS-1882/

Suggestions (48):
=====
* This release is more than 4 months old. Check the website or GitHub to see if there is an update available. [LYNIS]
  https://cisofy.com/lynis/controls/LYNIS/

* Install libpam-tmpdir to set $TMP and $TMPDIR for PAM sessions [DEB-0280]
  https://cisofy.com/lynis/controls/DEB-0280/

* Install apt-listbugs to display a list of critical bugs prior to each APT installation. [DEB-0010]
  https://cisofy.com/lynis/controls/DEB-0010/

* Install apt-listchanges to display any significant changes prior to any upgrade via APT. [DEB-0011]
  https://cisofy.com/lynis/controls/DEB-0011/

* Install needrestart, alternatively to debian-goodies, so that you can run needrestart after upgrades to determine which daemons are using old versions of libraries and need restarting. [DEB-0031]
  https://cisofy.com/lynis/controls/DEB-0031/

* Install fail2ban to automatically ban hosts that commit multiple authentication errors. [DEB-0000]
  https://cisofy.com/lynis/controls/DEB-0000/

* Set a password on GRUB boot loader to prevent altering boot configuration (e.g. boot in single user mode without password) [BOOT-5122]
  https://cisofy.com/lynis/controls/BOOT-5122/

* Consider hardening system services [BOOT-5204]
  Details : Run '/usr/bin/systemd-analyze security SERVICE' for each service
  https://cisofy.com/lynis/controls/BOOT-5204/

* If not required, consider explicit disabling of core dump in /etc/security/limits.conf file [KNML-5020]
  https://cisofy.com/lynis/controls/KNML-5020/

* Check PAM configuration, add rounds if applicable and expire passwords to encrypt with new values [AUTH-9229]
  https://cisofy.com/lynis/controls/AUTH-9229/

* Configure password hashing rounds in /etc/login.defs [AUTH-9230]
  https://cisofy.com/lynis/controls/AUTH-9230/

* When possible set expire dates for all password protected accounts [AUTH-9202]
  https://cisofy.com/lynis/controls/AUTH-9202/

```

Figure 21: Resumen de warnings y sugerencias identificadas durante la auditoría

El análisis identificó:

- **3 Warnings** (advertencias críticas):
 1. Paquetes vulnerables encontrados [PKGS-7392]
 2. Módulo iptables cargado sin reglas activas [FIRE-4512]
 3. Archivo de configuración Redis (/etc/redis/redis-openvas.conf) con permisos de lectura mundial [DBS-1882]
- **48 Suggestions** (sugerencias de mejora), entre las que destacan:
 - Actualizar Lynis a una versión más reciente
 - Instalar herramientas de monitoreo PAM (libpam-tmpdir)
 - Configurar utilidades para visualizar bugs críticos antes de instalaciones APT
 - Instalar needrestart o debian-goodies para detectar servicios que requieren reinicio
 - Implementar fail2ban para protección contra ataques de fuerza bruta
 - Establecer contraseña en el cargador de arranque GRUB
 - Endurecer servicios del sistema
 - Configurar rondas de hashing de contraseñas en PAM

4.3.5. Índice de Endurecimiento (Hardening Index)

Uno de los indicadores más importantes que proporciona Lynis es el **Hardening Index**, que cuantifica el nivel de seguridad del sistema en una escala de 0 a 100. La Figure 22 presenta este resultado.

```
=====
Lynis security scan details:

Hardening index : 56 [#####          ]
Tests performed : 256
Plugins enabled : 1

Components:
- Firewall           [V]
- Malware scanner    [X]

Scan mode:
Normal [V] Forensics [ ] Integration [ ] Pentest [ ]

Lynis modules:
- Compliance status  [?]
- Security audit      [V]
- Vulnerability scan  [V]

Files:
- Test and debug information : /var/log/lynis.log
- Report data                : /var/log/lynis-report.dat

=====

Lynis 3.0.9

Auditing, system hardening, and compliance for UNIX-based systems
(Linux, macOS, BSD, and others)

2007-2021, CISOfy - https://cisofy.com/lynis/
Enterprise support available (compliance, plugins, interface and tools)
=====
```

Figure 22: Índice de endurecimiento del sistema y resumen de la auditoría de seguridad

Los resultados obtenidos fueron:

- **Hardening index:** 56 de 100 posibles
- **Tests realizados:** 256 pruebas de seguridad ejecutadas
- **Plugins habilitados:** 1 plugin activo (Debian Tests)

El índice de 56 indica que el sistema presenta una configuración de seguridad **moderada**, con amplias oportunidades de mejora. Este valor es típico de instalaciones por defecto de Ubuntu sin hardening adicional aplicado.

Adicionalmente, el resumen muestra:

- **Firewall:** No configurado [X]
- **Malware scanner:** No instalado [X]
- **Modo de escaneo:** Normal (sin modo forense ni de integración)

Los archivos de resultados se generaron en:

- Registro detallado: /var/log/lynis.log
- Datos del reporte: /var/log/lynis-report.dat

4.3.6. Conclusiones del Caso Práctico

El análisis realizado por Lynis sobre el sistema Ubuntu 24.04 LTS revela un **nivel de seguridad moderado** con un hardening index de 56/100. Este resultado es consistente con una instalación estándar del sistema operativo sin aplicación de medidas de endurecimiento adicionales.

La auditoría realizada con Lynis demuestra la efectividad de esta herramienta para:

- Identificar sistemáticamente debilidades de configuración en sistemas operativos Unix/Linux
- Cuantificar objetivamente el nivel de seguridad mediante el hardening index
- Proporcionar recomendaciones específicas y accionables con documentación de referencia
- Facilitar el seguimiento del progreso en la implementación de controles de seguridad

5. OWASP ZAP (Zed Attack Proxy)

En el contexto actual de la seguridad de aplicaciones web, donde las vulnerabilidades representan uno de los vectores de ataque más explotados, resulta imprescindible contar con herramientas especializadas que permitan identificar y evaluar las debilidades de seguridad antes de que sean descubiertas por actores maliciosos. OWASP ZAP [5] se posiciona como una de las soluciones más completas y reconocidas para la auditoría de seguridad de aplicaciones web, siendo desarrollada y mantenida por la comunidad OWASP (Open Web Application Security Project).

OWASP ZAP es un escáner de seguridad de código abierto diseñado específicamente para encontrar vulnerabilidades en aplicaciones web durante las fases de desarrollo y pruebas. Funciona como un proxy intermediario entre el navegador del usuario y la aplicación web objetivo, permitiendo interceptar, inspeccionar y modificar el tráfico HTTP/HTTPS. Esta capacidad facilita tanto el análisis automatizado mediante escáneres activos y pasivos, como las pruebas manuales de penetración por parte de profesionales de seguridad.

Se clasifica dentro de las herramientas de: [6]

- Análisis de seguridad de aplicaciones web (DAST - Dynamic Application Security Testing).
- Pruebas de penetración y ethical hacking.
- Detección de vulnerabilidades OWASP Top 10.

5.1. Características Principales

La herramienta **OWASP ZAP** destaca por las siguientes funcionalidades clave:

1. **Código abierto y gratuito:** Disponible bajo licencia Apache 2.0, completamente gratuito sin limitaciones de funcionalidad.
2. **Multiplataforma:** Compatible con **Windows, Linux, macOS** y cualquier sistema que soporte Java.
3. **Proxy interceptor:** Actúa como intermediario entre el navegador y la aplicación web, permitiendo inspeccionar y modificar peticiones y respuestas HTTP/HTTPS en tiempo real.
4. **Escaneo automático:** Realiza análisis automatizados para detectar vulnerabilidades comunes incluyendo **XSS, SQL Injection, CSRF, XXE** y otras del OWASP Top 10.
5. **Análisis pasivo:** Examina el tráfico sin modificar las peticiones, identificando problemas de seguridad sin impacto en la aplicación.
6. **Spider (araña web):** Rastrea automáticamente la aplicación web para descubrir todas las páginas, parámetros y puntos de entrada disponibles.
7. **Fuzzing avanzado:** Permite realizar pruebas de fuzzing sobre parámetros específicos para identificar comportamientos inesperados o vulnerabilidades.
8. **Interfaz gráfica intuitiva:** Proporciona una GUI completa con visualización de resultados, árbol de sitios web y editor de peticiones.
9. **API REST y automatización:** Ofrece una API completa para integración en pipelines CI/CD y automatización de pruebas de seguridad.
10. **Marketplace de add-ons:** Extensible mediante complementos que añaden funcionalidades específicas como generación de reportes, integración con otras herramientas o técnicas de prueba especializadas.
11. **Generación de reportes:** Crea informes detallados en múltiples formatos (**HTML, XML, JSON, Markdown**) con clasificación de vulnerabilidades por severidad.

5.2. Ventajas y Desventajas

Entre sus ventajas, OWASP ZAP sobresale por su versatilidad y potencia, siendo adecuado tanto para usuarios principiantes como para profesionales experimentados en seguridad. Su capacidad para realizar tanto análisis automatizados como pruebas manuales avanzadas lo convierte en una herramienta integral que cubre todo el espectro de la auditoría de aplicaciones web. La comunidad activa de OWASP garantiza actualizaciones constantes, incorporando nuevas técnicas de detección para vulnerabilidades emergentes. Además, su integración nativa en entornos de desarrollo mediante API REST facilita la implementación de DevSecOps, permitiendo detectar vulnerabilidades desde las primeras fases del ciclo de vida del software.

No obstante, presenta algunas **limitaciones**: el escaneo automatizado puede generar falsos positivos que requieren validación manual por parte de analistas experimentados. En aplicaciones web complejas con JavaScript pesado o arquitecturas de una sola página (SPA), la efectividad del spider automático puede verse reducida, necesitando configuración adicional o exploración manual. Asimismo, el análisis activo puede generar una cantidad considerable de tráfico y peticiones modificadas que podrían afectar el rendimiento de aplicaciones en producción o activar sistemas de detección de intrusiones. A pesar de estas consideraciones, OWASP ZAP se mantiene como el estándar de facto en **auditoría de seguridad de aplicaciones web**, combinando potencia, flexibilidad, una comunidad global de soporte y la credibilidad del proyecto OWASP.

5.3. OWASP ZAP - Caso Práctico

5.3.1. Descripción del Caso

Este caso práctico realiza un análisis de seguridad web utilizando OWASP ZAP sobre el sitio `http://testphp.vulnweb.com/`, un entorno de pruebas diseñado para identificar vulnerabilidades del OWASP Top 10.

5.3.2. Entorno de Pruebas

- **Herramienta:** OWASP ZAP 2.16.1
- **Objetivo:** `http://testphp.vulnweb.com/`
- **Tipo de escaneo:** Automatizado (Spider + Active Scan)
- **Fecha de análisis:** 11 de noviembre de 2025

5.3.3. Inicialización de OWASP ZAP

Al iniciar OWASP ZAP, se presenta una pantalla de bienvenida con tres opciones principales (Figure 23):

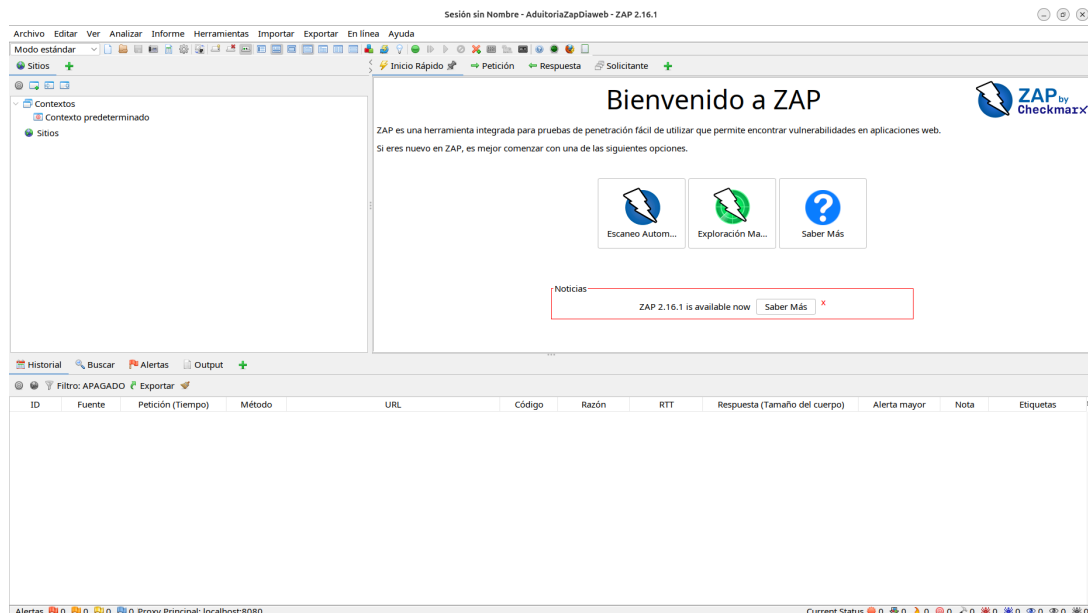


Figure 23: Pantalla de bienvenida de OWASP ZAP

- **Escaneo Automatizado:** Análisis rápido y completo de la aplicación web
- **Exploración Manual:** Análisis mediante proxy interceptor para pruebas personalizadas
- **Saber Más:** Acceso a documentación

Para este análisis se seleccionó el **Escaneo Automatizado**, que combina el rastreo y análisis de vulnerabilidades en un único proceso.

5.3.4. Configuración del Escaneo

La Figure 24 muestra la configuración del escaneo automatizado con los siguientes parámetros:

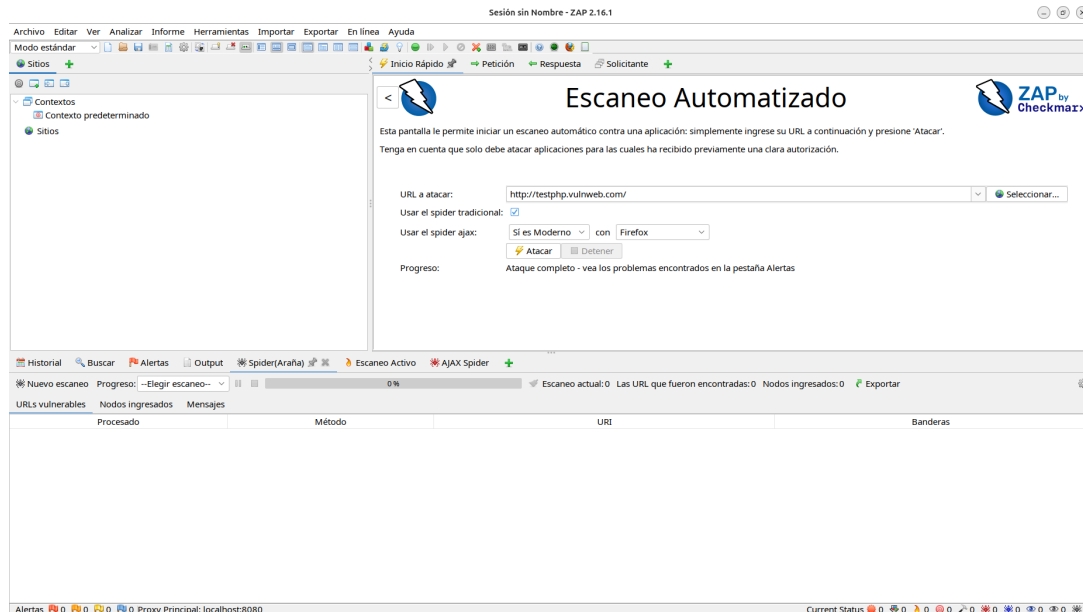


Figure 24: Configuración del escaneo automatizado

- **URL a atacar:** `http://testphp.vulnweb.com/`
- **Spider tradicional:** Habilitado para rastreo de enlaces HTML
- **Spider AJAX:** Configurado con navegador Firefox para detectar contenido JavaScript dinámico

5.3.5. Fase de Spider (Rastreo Web)

El spider explora la aplicación para descubrir todas las páginas y recursos disponibles (Figure 25).

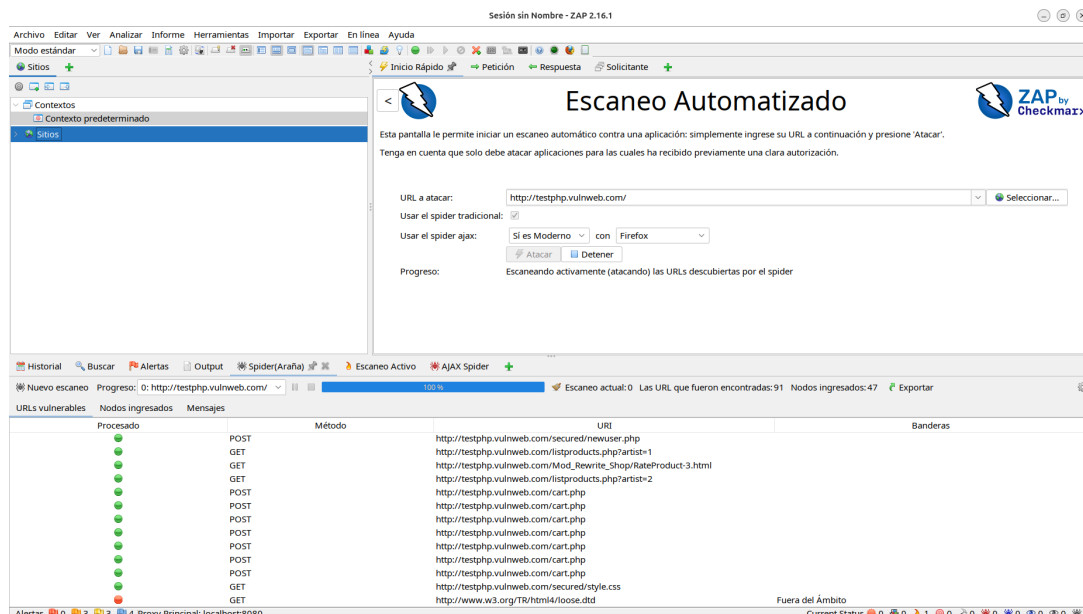


Figure 25: Progreso del spider durante el rastreo

Resultados del rastreo:

- **Progreso:** 100% completado
- **URLs encontradas:** 91 endpoints
- **Nodos ingresados:** 47 recursos únicos

Se identificaron endpoints importantes como:

- `/cart.php` - carrito de compras

- /listproducts.php - listado de productos
- /secured/newuser.php - registro de usuarios

5.3.6. Estructura del Sitio Web

Tras finalizar el spider, ZAP construye un árbol jerárquico de la aplicación (Figure 26).

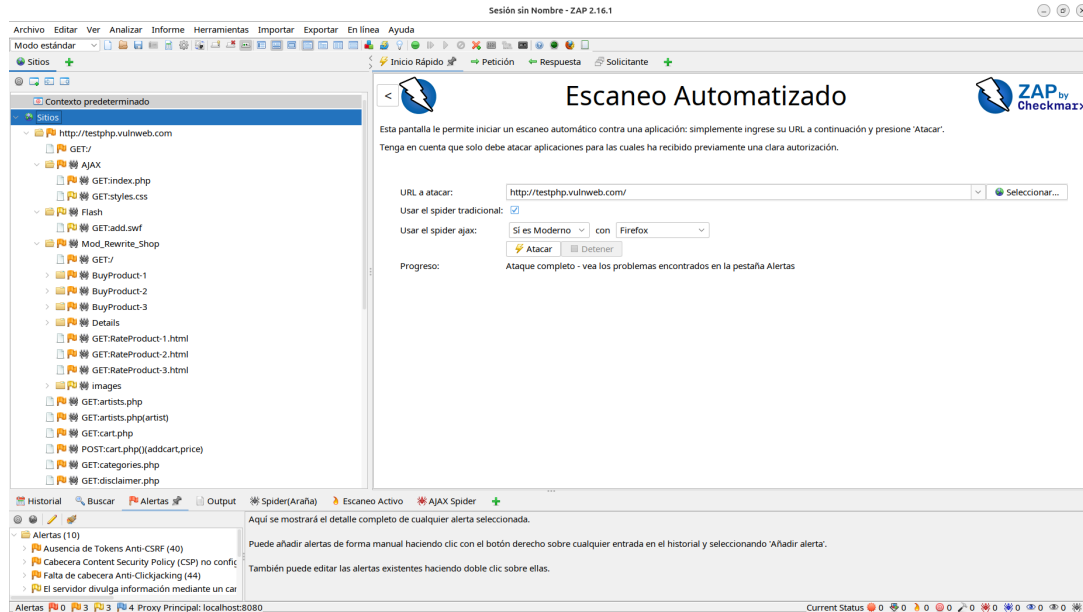


Figure 26: Árbol jerárquico del sitio web descubierto

Recursos principales identificados:

- Páginas de inicio y estilos CSS
- Funcionalidades AJAX y Flash
- Tienda con URLs optimizadas (/Mod_Rewrite_Shop/)
- Módulos de compra de productos
- Páginas de artistas, categorías y carrito

El panel inferior muestra las primeras alertas detectadas:

- Ausencia de Tokens Anti-CSRF (40 instancias)
- Falta de cabecera Content Security Policy
- Ausencia de protección Anti-Clickjacking (44 instancias)
- Divulgación de versión del servidor

5.3.7. Escaneo Activo de Vulnerabilidades

Después del descubrimiento, ZAP inicia el **Escaneo Activo**, enviando peticiones modificadas para identificar vulnerabilidades explotables (Figure 27).

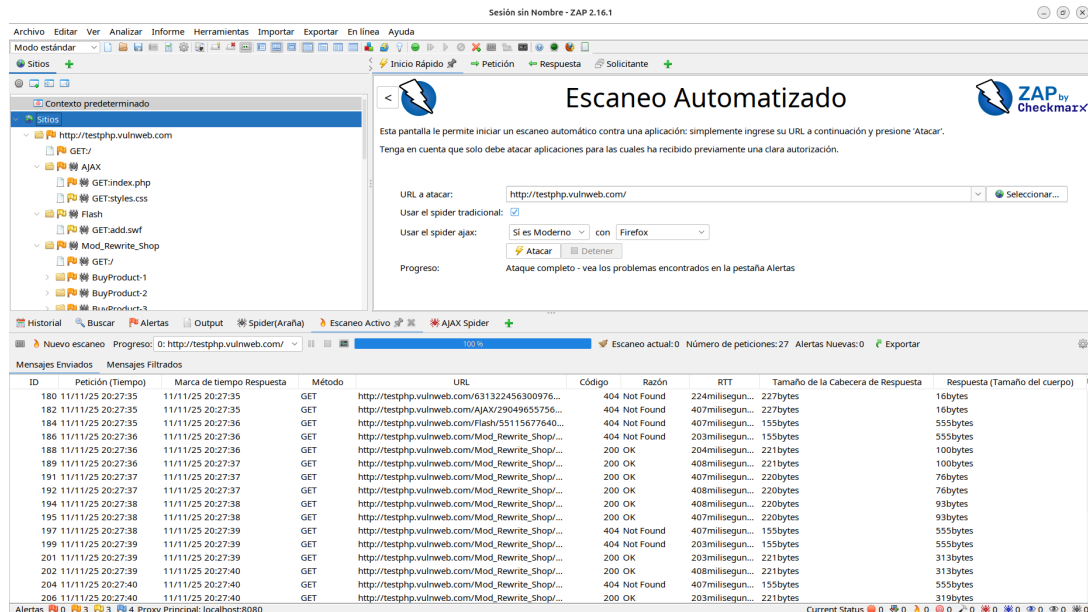


Figure 27: Escaneo activo en progreso

Detalles del escaneo:

- **Total de peticiones:** 27 peticiones HTTP
- **Métodos utilizados:** GET y POST
- **Respuestas exitosas (200 OK):** 15 peticiones
- **Recursos no encontrados (404):** 12 peticiones
- **Tiempo de respuesta:** 203-408 milisegundos

El escáner generó variaciones de parámetros para probar diferentes tipos de inyecciones y manipulaciones, mientras que las peticiones fuera del alcance fueron marcadas apropiadamente.

5.3.8. Análisis de Alertas Detectadas

Una vez completado el escaneo, ZAP consolida las vulnerabilidades en el panel de **Alertas** (Figure 28).

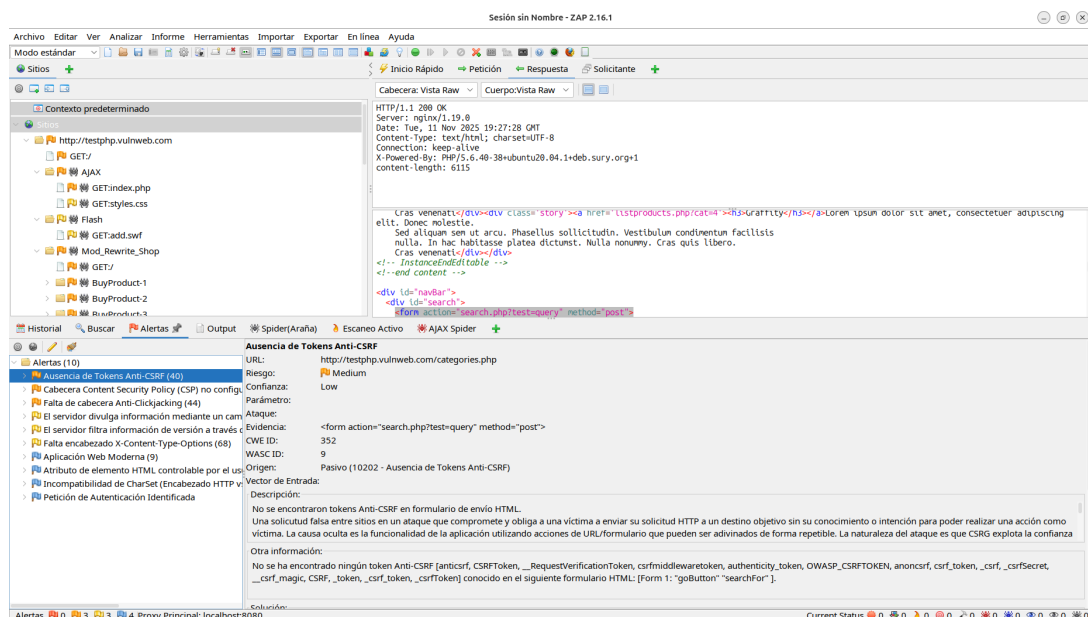


Figure 28: Panel de alertas clasificadas por severidad

Resumen de vulnerabilidades (10 tipos diferentes):

Severidad Media:

- **Ausencia de Tokens Anti-CSRF:** 40 instancias - permite ataques de falsificación de peticiones entre sitios

Severidad Baja:

- Falta de cabecera Content Security Policy (CSP)
- Ausencia de protección Anti-Clickjacking: 44 instancias
- Divulgación de versión del servidor: nginx/1.19.0
- Falta de cabecera X-Content-Type-Options: 68 instancias

Informativas:

- Atributos HTML controlables por el usuario
- Inconsistencias en codificación de caracteres
- Detección de formularios de autenticación

5.3.9. Generación del Informe de Auditoría

OWASP ZAP permite generar informes profesionales que documentan todos los hallazgos (Figure 29).

The screenshot shows the 'Generar Informe' window with the following configuration:

- Título de Informe:** ZAP por Informe de Escaneo Checkmarx
- Nombre de Informe:** 2025-11-11-ZAP-Report-.html
- Directorio de Informe:** /home/miguel
- Descripción:** Auditoria Prueba
- Contexto:** Contexto predeterminado
- Sitios:** http://testphp.vulnweb.com
- Generar si no hay alertas:** ☐
- Informe a Mostrar:** ☒

Buttons at the bottom: Generar Informe, Reiniciar, Cancelar.

Figure 29: Configuración de generación de informe HTML

Configuración del informe:

- **Título:** “ZAP por Informe de Escaneo Checkmarx”
- **Nombre de archivo:** 2025-11-11-ZAP-Report-.html
- **Directorio:** /home/miguel

- **Descripción:** “Auditoria Prueba”
- **Sitio objetivo:** <http://testphp.vulnweb.com>

Esta funcionalidad es fundamental para documentar formalmente las auditorías, cumplimiento normativo y comunicación de hallazgos a los equipos de desarrollo.

5.3.10. Informe HTML Generado

El informe HTML proporciona una presentación profesional de los hallazgos (Figure 30).

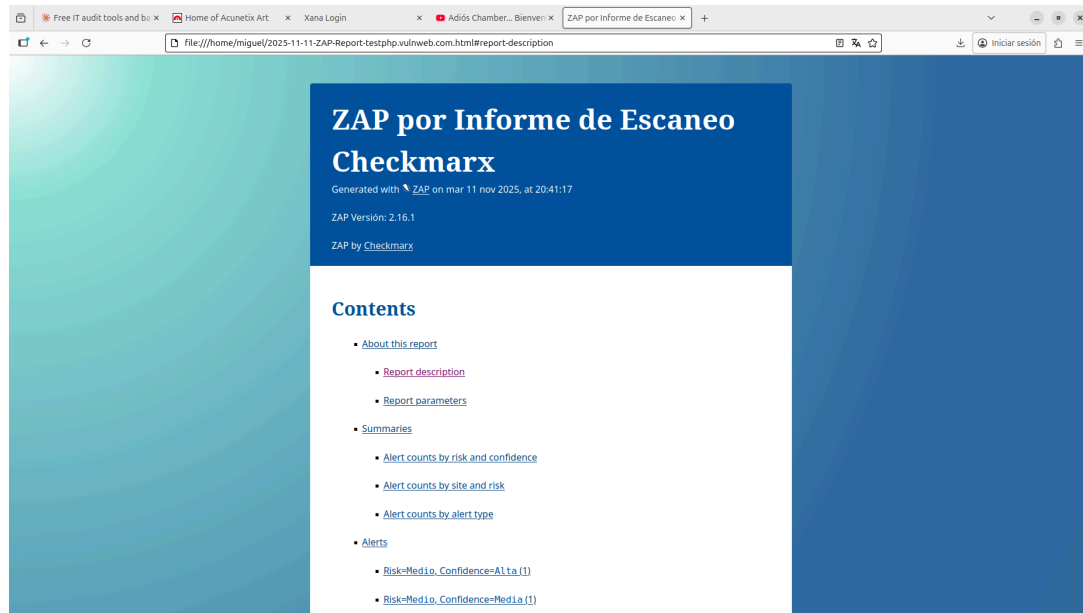


Figure 30: Portada del informe HTML con tabla de contenidos

Estructura del informe:

- Información general del análisis
- Parámetros de configuración utilizados
- Resúmenes de vulnerabilidades:
 - Distribución por riesgo y confianza
 - Alertas agrupadas por sitio y severidad
 - Clasificación por tipo de vulnerabilidad
- Detalle exhaustivo de cada hallazgo

El diseño profesional con degradado azul facilita la lectura y presentación a diferentes audiencias técnicas y no técnicas.

Bibliografía

- [1] Wireshark Foundation, *Wireshark Network Protocol Analyzer: Official Documentation*. Reno, Nevada, USA: Wireshark Foundation, 2024. [Online]. Available: <https://www.wireshark.org/>
- [2] R. Bejtlich, *The Practice of Network Security Monitoring: Understanding Incident Detection and Response*. San Francisco, USA: No Starch Press, 2013.
- [3] CISOfy, *Lynis: Security Auditing Tool for Unix/Linux Systems*. Netherlands: CISOfy, 2024. [Online]. Available: <https://cisofy.com/lynis/>
- [4] J. Turnbull, *The Art of Monitoring*. New York, USA: Turnbull Press, 2016.
- [5] OWASP Foundation, *OWASP Zed Attack Proxy (ZAP): Official Documentation*. Wakefield, USA: OWASP Foundation, 2024. [Online]. Available: <https://www.zaproxy.org/>
- [6] D. Stuttard and M. Pinto, *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws*, 2nd ed. Indianapolis, USA: Wiley, 2011.